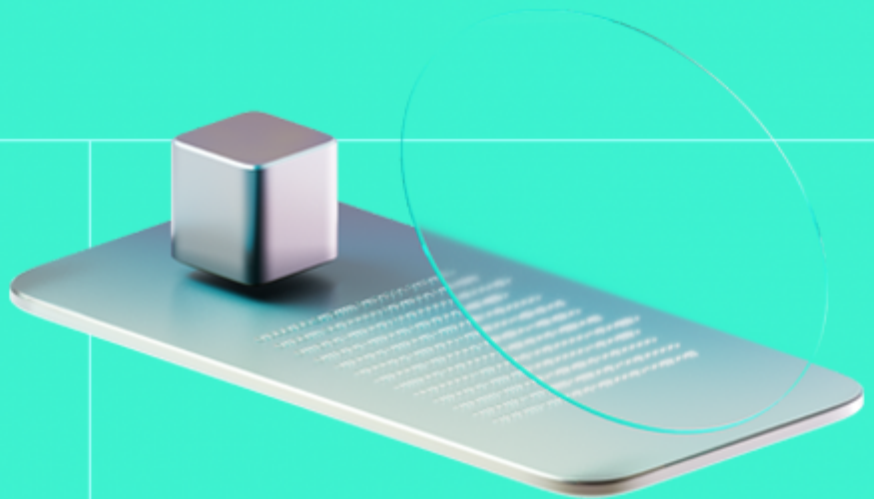




Smart Contract Code Review And Security Analysis Report

Customer: Knoxnet

Date: 15/04/2026



We express our gratitude to the Knoxnet team for the collaborative engagement that enabled the execution of this Smart Contract Security Assessment.

KnoxNet is a fee-on-transfer token architecture built on a single ERC-20 contract, featuring built-in trading controls, tax handling, blacklist capabilities, and automated fee conversion through Uniswap V2.

Document

Name	Smart Contract Code Review and Security Analysis Report for Knoxnet
Audited By	Olesia Bilenka
Approved By	Seher Saylik
Website	https://www.knoxnet.io/
Changelog	13/04/2026 - Preliminary Report
	15/04/2026 - Final Report
Platform	Ethereum
Language	Solidity
Tags	Fungible Token
Methodology	https://docs.hacken.io/methodologies/smart-contracts

Review Scope

Repository	https://github.com/Knoxnetofficial/knoxnet-contracts
Commit	748a360
Remediation Commit	42b580b

Audit Summary

The system users should acknowledge all the risks summed up in the risks section of the report

42	32	6	4
Total Findings	Resolved	Accepted	Mitigated

Findings by Severity

Severity	Count
Critical	0
High	3
Medium	6
Low	12

Vulnerability	Severity	Status
F-2026-15965 - Incomplete Blacklist Enforcement in transferFrom Allows Blacklisted Callers to Bypass Transfer Restrictions	High	Fixed
F-2026-15966 - Excessive Initial Sell Tax Can Severely Restrict Exits and Is Inconsistent With the Contract's Tax Limit Invariants	High	Fixed
F-2026-15969 - Unvalidated taxDenominator Breaks Tax Cap Invariants and Causes Hardcoded Limits to Misapply in Both Directions	High	Fixed
F-2026-15970 - Missing Transfer Event for Taxed Amount Breaks ERC20 Compliance and Causes Balance Tracking Mismatches	Medium	Fixed
F-2026-15974 - Blacklist Enforcement Bypassed When Recipient Is Transaction-Limit Exempt	Medium	Fixed
F-2026-15975 - transferOwnership Does Not Update Privileged Exemptions and Router Allowances After Ownership Transfer	Medium	Fixed
F-2026-15980 - Fee Distribution in _autoSwapBack Uses Configured Tax Rates as Proxy for Actual Fee Composition, Causing Systematic Misallocation Between Marketing and Liquidity	Medium	Fixed
F-2026-15984 - Incomplete Transfer Classification Causes Inconsistent Limit and Tax Behavior for Wallet-to-Wallet Transfers	Medium	Mitigated
F-2026-16037 - Auto-Swap Reverts During Sell Execution Can Deny User Exits	Medium	Fixed
F-2026-15971 - Zero Slippage Protection in Swap Functions Exposes Tax Conversions to Sandwich Attacks	Low	Accepted
F-2026-15972 - Incorrect blacklistCount Accounting on Duplicate Entries Causes Persistent State Inconsistency	Low	Fixed

Vulnerability	Severity	Status
F-2026-15973 - airdrop Bypasses Transfer Restrictions and Allows Unrestricted Token Distribution	Low	Mitigated
F-2026-15977 - Lack of Timelock on Critical Admin Functions Enables Immediate High-Impact Configuration Changes	Low	Accepted
F-2026-15978 - Wallet Limit Validation Uses Gross Transfer Amount and Can Reject Transfers That Would Satisfy the Post-Tax Limit	Low	Fixed
F-2026-15981 - Limited On-Chain Visibility of Tax Configuration Obscures Effective Tax State	Low	Fixed
F-2026-15985 - Non-Reentrant-Safe swapping Modifier Can Be Corrupted Through Nested Execution via Marketing Receiver Callback	Low	Fixed
F-2026-15996 - Implicit Infinite Router Allowance in the Constructor Grants Spending Authority Over the Owner Balance	Low	Mitigated
F-2026-16002 - manualClogClearPercent Ignores the Configured Liquidity Split and Routes All Swap Proceeds to the Marketing Receiver	Low	Fixed
F-2026-16003 - Zero Slippage Protection in addLiquidityETH Allows Liquidity to Be Added at Adversarially Shifted Pool Ratios	Low	Fixed
F-2026-16005 - Missing Lower Bounds on Transaction and Wallet Limits Allow Transfers to Be Effectively Frozen	Low	Fixed
F-2026-16009 - isTxLimitExempt Simultaneously Disables Both Transaction and Wallet Limits	Low	Fixed
F-2026-15976 - Missing Zero-Address Validation in setTaxReceivers Can Permanently Misroute Tax Proceeds and Liquidity Assets	Info	Fixed
F-2026-15979 - Use of tx.origin in Blacklist Event Produces Unreliable Caller Attribution	Info	Fixed
F-2026-15982 - Missing Zero-Address Validation in airdrop Allows Silent Token Burning and Distorts Circulating Supply Reporting	Info	Fixed
F-2026-15983 - Inconsistent Amount Units Across Administrative and View Functions Increase the Risk of Off-Chain Misinterpretation and Misconfiguration	Info	Accepted
F-2026-15986 - Mid-Section State Variable Declaration Reduces Structural Consistency	Info	Fixed
F-2026-15987 - Missing Events for Administrative Configuration Changes Reduces On-Chain Observability	Info	Fixed
F-2026-15988 - Co-Locating All Contracts and Interfaces in a Single Source File Reduces Reviewability and Maintainability	Info	Accepted
F-2026-15989 - Unused using Address for address Declaration Introduces Dead Code and Reduces Code Clarity	Info	Fixed
F-2026-15990 - Non-Mutable Configuration Values Are Stored in Regular Storage Instead of constant or immutable	Info	Fixed
F-2026-15991 - Unnamed WalletRestricted Event Parameters Reduce Log Readability and Off-Chain Decoding Clarity	Info	Fixed
F-2026-15992 - Missing Input Validation in recoverStuckETH Can Cause	Info	Fixed

Vulnerability	Severity	Status
Misconfiguration Reverts or Irrecoverable ETH Transfers		
F-2026-15993 - Redundant amount > 0 Condition in Auto-Swap Trigger Adds Unnecessary Logic	Info	Fixed
F-2026-15994 - Inconsistent Use of _msgSender() and msg.sender Reduces Call-Context Consistency	Info	Fixed
F-2026-15995 - Public Visibility of _shouldApplyTax Conflicts With Internal Naming Convention and Reduces Interface Clarity	Info	Fixed
F-2026-15998 - Abbreviated Parameter Names Reduce Interface Clarity and Code Readability	Info	Fixed
F-2026-15999 - Equal Token Split for Liquidity Addition Produces Suboptimal Pairing After Price Impact From the Preceding Swap	Info	Accepted
F-2026-16001 - Missing Cross-Parameter Validation in configureAutoSwap Allows Inefficient Auto-Swap Configurations	Info	Fixed
F-2026-16004 - Using block.timestamp as the Router Deadline Removes Effective Expiry Protection	Info	Fixed
F-2026-16006 - Widespread Use of Magic Numbers Reduces Configuration Clarity and Auditability	Info	Mitigated
F-2026-16007 - Multiple Percentage Conventions in the Same Contract Reduce Numeric Consistency and Increase Configuration Risk	Info	Accepted
F-2026-16008 - Single-Step Ownership Transfer Increases the Risk of Irrecoverable Administrative Misassignment	Info	Fixed

Documentation quality

- The project does not provide a specification describing the intended behavior of trading controls, fee processing, blacklist logic, launch flow, or administrative configuration boundaries.
- A technical design description is not provided. In particular, there is no separate documentation explaining how tax collection, auto-swap, liquidity support, privileged roles, and transfer restrictions are expected to interact under different configurations.

Code quality

- The code inlines commonly known utility and base-contract logic instead of reusing external well-established dependencies through imports. This increases file size and reduces modularity and review clarity.
- Several template-style patterns are present, particularly around standard ERC-20, ownership, and router-integration logic, combined directly with custom business logic in a single file. This makes it harder to distinguish standard behavior from protocol-specific behavior during review and maintenance.
- The development environment is configured.

Test coverage

Code coverage of the project is **52.10%** (branch coverage).

- The test suite covers deployment, two-step ownership transfer, ERC-20 core behavior, trading activation, buy and sell transaction limits, wallet-limit enforcement, blacklist add flow, tax application, tax configuration updates, manual clog clearing, airdrop behavior, ETH recovery, liquidity-pool management, and auto-swap configuration setters.
- The main remaining coverage gap is the automatic `_autoSwapBack` execution path during live sell transactions, including swap execution, liquidity addition, per-block auto-swap counting, and related `try/catch` failure branches.
- Additional uncovered paths include blacklist removal, recipient and caller blacklist reverts, the `inSwap` internal transfer bypass, several view/helper functions, and direct execution of the `_tryAddLiquidityETH` helper.

Table of Contents

System Overview	8
Privileged Roles	8
Potential Risks	10
Findings	11
Vulnerability Details	11
Disclaimers	117
Appendix 1. Definitions	118
Severities	118
Potential Risks	118
Appendix 2. Scope	119
Appendix 3. Additional Valuables	120

System Overview

KnoxNet is a fee-on-transfer token protocol built around a single main ERC-20 token contract with integrated trading controls, tax collection, blacklist management, and automated fee conversion through Uniswap V2.

It consists of the following components:

KnoxNet — the main ERC-20 token contract that mints the full initial supply to the deployer at deployment and manages all token logic internally, including transfer taxation, transaction and wallet limits, blacklist restrictions, launch controls, and fee processing.

It has the following attributes:

Name: KnoxNet

Symbol: KNX

Decimals: 18

Total supply: 1b tokens

The protocol is designed for Ethereum-compatible deployment and is tightly coupled to a Uniswap V2-style trading flow. The contract creates its own liquidity pair against WETH during deployment and distinguishes between buy, sell, and regular transfer activity based on whether a liquidity pool is involved.

The token applies configurable fees on buy and sell transactions, with optional taxation on wallet-to-wallet transfers. Collected fee tokens are accumulated inside the contract and can later be converted into ETH. Depending on configuration, the converted value may be routed to a marketing receiver and, if liquidity fees are enabled, partially paired back into liquidity.

The system includes multiple administrative controls. The owner can adjust tax settings, transaction and wallet limits, fee receivers, liquidity-pool classification, and auto-swap behavior. A separate trusted-operator role can perform selected sensitive actions such as opening trading, updating the blacklist, and recovering ETH from the contract.

The protocol does not include staking, vesting, governance, upgradeability, or minting beyond the initial supply. Its design centers on token trading controls and post-trade fee handling rather than on broader DeFi utility. **KnoxNet** is a fee-on-transfer token protocol built around a single main ERC-20 token contract with integrated trading controls, tax collection, blacklist management, and automated fee conversion through Uniswap V2.

Privileged roles

- The owner of the **KnoxNet** contract role can arbitrarily change taxation parameters, transaction and wallet restrictions, fee receivers, transfer-tax behavior, liquidity-pool classification, and auto-swap settings. The owner can therefore materially change core token behavior and trading conditions at will.
- The owner can also assign or remove trusted operators. Through this capability, the owner indirectly controls which external addresses are allowed to perform selected sensitive operational actions on the protocol.

- A trusted operator of the **KnoxNet** contract has elevated operational authority over a subset of critical functions. This role can enable trading, update the blacklist, and recover ETH from the contract to an arbitrary address. As a result, trusted operators are able to influence market access and extract ETH held by the contract.
- The owner and trusted operators are exempt from some of the restrictions imposed on regular users, either directly or through configurable exemption mappings. This creates a privileged administrative layer with materially different execution rights from ordinary token holders.

Potential Risks

- The **KnoxNet** contract performs automated token-to-ETH conversions through the Uniswap V2 router without oracle-based price validation or minimum output enforcement, exposing every auto-swap and manual clog-clear operation to sandwich attacks, MEV extraction, and execution at manipulated spot prices.
- The **KnoxNet** contract grants the owner unrestricted ability to modify tax rates, transaction limits, wallet limits, tax receivers, auto-swap parameters, exemption flags, blacklist entries, and liquidity pool designations at any time without timelocks or multi-signature requirements, concentrating full economic control in a single address.
- The **KnoxNet** contract allows the owner to designate arbitrary external addresses as liquidity pools via `addLiquidityPool`, which changes how the tax engine classifies transfers to and from those addresses, potentially altering tax behavior for any wallet without the affected user's awareness.
- The **KnoxNet** contract relies on a single trusted operator role that, once granted, retains permanent blacklist and ETH withdrawal capabilities even after ownership renunciation, with no on-chain mechanism to revoke the role if the private key is compromised.
- The **KnoxNet** contract does not impose minimum thresholds on transaction limits or wallet caps, allowing the owner to set values low enough to effectively freeze all non-exempt trading activity without disabling the contract.
- The **KnoxNet** contract accumulates tax tokens internally and converts them to ETH during user sell transactions, meaning a temporary failure or manipulation of the Uniswap V2 pair could cause user sells to revert or execute with degraded economic outcomes.
- The **KnoxNet** contract sends ETH to the marketing receiver via a low-level call with silent failure handling during auto-swap, meaning if the receiver becomes unable to accept ETH, proceeds silently accumulate in the contract with no automated retry or alerting mechanism.
- The **KnoxNet** contract pairs accumulated liquidity tokens with ETH and adds them to the Uniswap V2 pool with zero slippage protection, allowing arbitrageurs to extract value from the liquidity addition if pool reserves are temporarily skewed.
- The **KnoxNet** contract does not emit events for the majority of its administrative state changes, making it difficult for off-chain monitoring systems, governance dashboards, or community watchdogs to detect and react to potentially harmful configuration updates in real time.
- The **KnoxNet** contract uses the `airdrop` function with `_basicTransfer`, which bypasses blacklist checks, wallet limits, transaction limits, and tax logic, meaning the owner can distribute tokens to any address regardless of its restriction status.

Findings

Vulnerability Details

[F-2026-15965](#) - Incomplete Blacklist Enforcement in transferFrom Allows Blacklisted Callers to Bypass Transfer Restrictions - High

Description:

In **KnoxNet**, blacklist enforcement is applied only to the `sender` address and does not cover either `msg.sender` or `recipient`. As a result, a blacklisted address can still initiate transfers through `transferFrom` when acting as an approved spender for a non-blacklisted account. The blacklist mechanism therefore fails to prevent blocked actors from continuing to move tokens.

In **KnoxNet**, the `transferFrom` function forwards execution to `_transferFrom`, which in turn applies blacklist checks through `_enforceTxLimit`. However, only the `sender` parameter is checked:

```
function transferFrom(
    address sender,
    address recipient,
    uint256 amount
) external override returns (bool) {
    // ...
    return _transferFrom(sender, recipient, amount);
}
// ...
function _enforceTxLimit(
    address sender,
    address recipient,
    uint256 amount
) internal view {
    // ...
    require(blacklist[sender] == 0, "Wallet blacklisted!");
}
```

This logic assumes that the token owner and the transaction initiator are the same actor. That assumption does not hold for `transferFrom`, where `msg.sender` is the spender. A blacklisted address can therefore obtain allowance from a non-blacklisted wallet and continue transferring or selling that wallet's tokens. Since `recipient` is not checked either, blacklisted addresses may still receive tokens.

Blacklisted addresses remain able to operate through the allowance flow and are not effectively excluded from token activity. The anti-bot and

administrative restriction mechanism can therefore be bypassed in practice. Blacklisted recipients may also continue accumulating tokens.

Assets:

- contracts/KnoxNet.sol [<https://github.com/Knoxnetofficial/knoxnet-contracts>]

Status:

Fixed

Classification

Impact Rate:	5/5
Likelihood Rate:	3/5
Exploitability:	Independent
Complexity:	Simple
Severity:	High

Recommendations

Remediation:

Blacklist validation should be applied to all relevant actors in the transfer path: `sender`, `recipient`, and `msg.sender`.

```
function _enforceTxLimit(
    address sender,
    address recipient,
    uint256 amount
) internal view {
    if (isTxLimitExempt[sender] || isTxLimitExempt[recipient]) return;
    require(
        amount <= (liquidityPools[sender] ? _maxBuyTxAmount : _maxSellTxAmount
    ),
        "Amount exceeds the tx limit."
    );
    require(blacklist[sender] == 0, "Sender blacklisted!");
    require(blacklist[recipient] == 0, "Recipient blacklisted!");
    require(blacklist[msg.sender] == 0, "Caller blacklisted!");
}
```

Resolution:

Fixed in `c20e980` :

In **KnoxNet**, blacklist validation now covers all relevant actors in the transfer path. The `_enforceTxLimit` function checks `sender`, `recipient`, and `_msgSender`, which prevents a blacklisted spender from using `transferFrom`

on behalf of a non-blacklisted holder and prevents transfers to blacklisted recipients. The allowance-based blacklist bypass described in the finding is therefore no longer present.

```
function _enforceTxLimit(
    address sender,
    address recipient,
    uint256 amount
) internal view {
    require(blacklist[sender] == 0, "Sender blacklisted!");
    require(blacklist[recipient] == 0, "Recipient blacklisted!");
    require(blacklist[_msgSender()] == 0, "Caller blacklisted!");
    if (isTxLimitExempt[sender] || isTxLimitExempt[recipient]) return;
    TransferType transferType = _classifyTransfer(sender, recipient);
    uint256 maxTxAmount = transferType == TransferType.Sell
        ? _maxSellTxAmount
        : _maxBuyTxAmount;
    require(
        amount <= maxTxAmount,
        "Amount exceeds the tx limit."
    );
}
```

Evidences

PoC 1: Blacklisted address bypasses blacklist as msg.sender

Reproduce:

```
function test_blacklisted_bypassesAsMsgSender_viaTransferFrom() public {
    clean.doApprove(
        address(token),
        address(blacklisted),
        type(uint256).max
    );

    token.updateBlacklist(_toArray(address(blacklisted)), true);

    vm.expectRevert("Wallet blacklisted!");
    blacklisted.doTransfer(
        address(token),
        address(recipient),
        1_000 * 1e18
    );
}
```

```

uint256 recipientBefore = token.balanceOf(address(recipient));

blacklisted.doTransferFrom(
    address(token),
    address(clean),
    address(recipient),
    10_000 * 1e18
);

uint256 recipientAfter = token.balanceOf(address(recipient));

assertTrue(
    recipientAfter > recipientBefore,
    "Blacklisted caller moved tokens via transferFrom - blacklist bypassed"
);
}

```

Results:

[PASS] test_blacklisted_bypassesAsMsgSender_viaTransferFrom() (gas: 179715)

PoC 2: Blacklisted address can still receive tokens

Reproduce:

```

function test_blacklisted_canStillReceiveTokens() public {
    token.updateBlacklist(_toArray(address(blacklisted)), true);

    uint256 balBefore = token.balanceOf(address(blacklisted));

    clean.doTransfer(address(token), address(blacklisted), 10_000 * 1e18)
;

    uint256 balAfter = token.balanceOf(address(blacklisted));

    assertTrue(
        balAfter > balBefore,
        "Blacklisted address received tokens - recipient not checked"
    );
}

```

```
emit log_named_uint(  
    "Tokens received by blacklisted address",  
    (balAfter - balBefore) / 1e18  
);  
}
```

Results:

[PASS] test_blacklisted_canStillReceiveTokens() (gas: 110772)

[F-2026-15966](#) - Excessive Initial Sell Tax Can Severely Restrict Exits and Is Inconsistent With the Contract's Tax Limit Invariants - High

Description:

In **KnoxNet**, the initial sell tax is configured to 90%, which causes sellers to retain only a small portion of the transferred amount. This materially restricts exit liquidity under the initial deployment state. The initial configuration is also inconsistent with the contract's own tax limit assumptions, because subsequent administrative updates enforce a 65% maximum sell tax while the initial state exceeds that threshold.

In **KnoxNet**, the initial tax configuration is defined as follows:

```
uint256 marketingBuyTaxBps = 500;
uint256 marketingSellTaxBps = 9000;
uint256 liquidityBuyTaxBps = 0;
uint256 liquiditySellTaxBps = 0;
uint256 totalBuyTaxBps = marketingBuyTaxBps + liquidityBuyTaxBps;
uint256 totalSellTaxBps = marketingSellTaxBps + liquiditySellTaxBps;
uint256 taxDenominator = 10000;
```

Under these values, the effective sell tax is $9000 / 10000 = 90\%$. The sell-side deduction is applied in `_applyTax`, which routes the taxed amount to the contract balance and leaves only the remainder for the recipient:

```
function _applyTax(
    address recipient,
    uint256 amount
) internal returns (uint256) {
    bool selling = liquidityPools[recipient];
    uint256 taxAmount = (amount * getTotalTaxBps(selling)) / taxDenominator;
    _balances[address(this)] += taxAmount;
    return amount - taxAmount;
}
```

This initial state does not align with the constraints enforced in `setTaxConfig`, where the combined sell tax is limited to 65%:

```
require(
    _liquiditySellTaxBps + _marketingSellTaxBps <= 6500,
    "Sell tax cannot exceed 65%"
);
```

As a result, the deployed parameters do not satisfy the same invariant that is later applied to administrative updates.

Under the initial configuration, sellers receive only 10% of the gross sell amount. This can make exits economically impractical until the tax parameters are changed. The inconsistency between the deployment state and the runtime tax cap may also invalidate assumptions made by users, reviewers, or integrations about the maximum configured sell tax.

Assets:

- contracts/KnoxNet.sol [<https://github.com/Knoxnetofficial/knoxnet-contracts>]

Status:

Fixed

Classification

Impact Rate:	5/5
Likelihood Rate:	5/5
Exploitability:	Semi-Dependent
Complexity:	Simple
Severity:	High

Recommendations

Remediation:

- The initial tax configuration should be aligned with the same bounds enforced for later updates. Deployment-time values should satisfy the intended sell-tax maximum from the outset.
- A single invariant should be maintained across both initialization and `setTaxConfig`, ensuring that the effective buy and sell tax values remain within the intended limits at all times.

Resolution:

Fixed in `c20e980` :

In the current **KnoxNet** implementation, the initial sell tax has been reduced from `9000` basis points to `500` basis points, which removes the 90% sell-side deduction present in the original deployment configuration. The initial values now remain within the same 65% bound enforced by `setTaxConfig`, so the deployment state is consistent with the administrative tax cap.

```
uint256 public marketingBuyTaxBps = 500;  
uint256 public marketingSellTaxBps = 500;
```

Evidences

PoC 1: Initial sell tax is 90% — verify default values

Reproduce:

```
function test_initialSellTax_is90Percent() public view {
    uint256 sellTax = token.getTotalTaxBps(true);
    uint256 buyTax = token.getTotalTaxBps(false);

    assertEq(sellTax, 9000, "Default sell tax should be 9000 bps (90%)");
    assertEq(buyTax, 500, "Default buy tax should be 500 bps (5%)");
}
```

Results:

[PASS] test_initialSellTax_is90Percent() (gas: 11102)

PoC 2: setTaxConfig enforces 65% cap — initial state violates it

Reproduce:

```
function test_setTaxConfig_rejectsCurrentSellTax() public {
    uint256 currentSellTax = token.getTotalTaxBps(true);
    assertEq(currentSellTax, 9000, "Current sell tax is 9000");

    vm.expectRevert("Sell tax cannot exceed 65%");
    token.setTaxConfig(0, 0, 500, 9000, 10000);
}
```

Results:

[PASS] test_setTaxConfig_rejectsCurrentSellTax() (gas: 14489)

PoC 3: User buys at 5%, sells at 90% — massive asymmetric loss

Reproduce:

```
function test_buyAndSell_asymmetricLoss() public {
    token.configureAutoSwap(false, 1000, 0);

    uint256 ethInvested = 1 ether;

    address[] memory buyPath = new address[](2);
    buyPath[0] = router.WETH();
    buyPath[1] = address(token);
}
```

```

vm.prank(address(user));
router.swapExactETHForTokensSupportingFeeOnTransferTokens{
    value: ethInvested
}(0, buyPath, address(user), block.timestamp + 3600);

uint256 tokensReceived = token.balanceOf(address(user));
assertTrue(tokensReceived > 0, "User should receive tokens");

uint256 ethBefore = address(user).balance;

user.sell(router, address(token), tokensReceived);

uint256 ethAfter = address(user).balance;
uint256 ethRecovered = ethAfter - ethBefore;

uint256 lossBps = ((ethInvested - ethRecovered) * 10000) / ethInvested;

assertTrue(
    lossBps > 9000,
    "User should lose >90% due to buy tax + sell tax + price impact"
);
}

```

Results:

```
[PASS] test_buyAndSell_asymmetricLoss() (gas: 252667)
```

[F-2026-15969](#) - Unvalidated taxDenominator Breaks Tax Cap Invariants and Causes Hardcoded Limits to Misapply in Both Directions - High

Description:

In **KnoxNet**, the `taxDenominator` value is owner-configurable but is not validated against the configured tax rates. The contract enforces hardcoded tax caps using absolute values such as `6500`, while the actual effective tax is computed relative to `taxDenominator`. As a result, the stated buy and sell limits no longer represent fixed percentage bounds and may become either materially weaker or materially stricter than intended.

In **KnoxNet**, `setTaxConfig` allows arbitrary updates to `taxDenominator`:

```
function setTaxConfig(
    uint256 _liquidityBuyTaxBps,
    uint256 _liquiditySellTaxBps,
    uint256 _marketingBuyTaxBps,
    uint256 _marketingSellTaxBps,
    uint256 _taxDenominator
) external onlyOwner {
    require(
        _liquidityBuyTaxBps + _marketingBuyTaxBps <= 6500,
        "Buy tax cannot exceed 65%"
    );
    require(
        _liquiditySellTaxBps + _marketingSellTaxBps <= 6500,
        "Sell tax cannot exceed 65%"
    );
    // ...
    taxDenominator = _taxDenominator;
}
```

However, the actual tax charged in **KnoxNet** is derived from the configured denominator:

```
function _applyTax(
    address recipient,
    uint256 amount
) internal returns (uint256) {
    bool selling = liquidityPools[recipient];
    uint256 taxAmount = (amount * getTotalTaxBps(selling)) / taxDenominator;
    _balances[address(this)] += taxAmount;
    return amount - taxAmount;
}
```

This creates a broken invariant. If `taxDenominator` is set below `10000`, the hardcoded `6500` cap permits a higher effective rate than 65%, including 100% tax when `taxDenominator == 6500`, and a reverting transfer path when the tax sum exceeds the denominator. If `taxDenominator` is set above `10000`, the same hardcoded cap becomes stricter than intended, because `6500 / taxDenominator` produces an effective rate below 65%. The limit checks are therefore decoupled from the value actually used in tax calculation.

The effective buy and sell tax may exceed the stated 65% cap or become inconsistent with administrative expectations, depending on the configured denominator. In the lower-denominator case, taxed transfers may become confiscatory or revert entirely. In the higher-denominator case, the enforced limits no longer correspond to the percentages described by the contract.

Assets:

- `contracts/KnoxNet.sol` [<https://github.com/Knoxnetofficial/knoxnet-contracts>]

Status:

Fixed

Classification

Impact Rate:	5/5
Likelihood Rate:	5/5
Exploitability:	Semi-Dependent
Complexity:	Simple
Severity:	High

Recommendations

Remediation:

`taxDenominator` should either be fixed to `10000` or validated so that all tax cap checks are normalized to the same denominator used in `_applyTax`. Hardcoded bounds should not be compared directly against tax values when the denominator is mutable.

```
function setTaxConfig(
    uint256 _liquidityBuyTaxBps,
    uint256 _liquiditySellTaxBps,
    uint256 _marketingBuyTaxBps,
    uint256 _marketingSellTaxBps,
    uint256 _taxDenominator
) external onlyOwner {
    require(_taxDenominator == 10000, "Invalid tax denominator");
}
```

```

require(
    _liquidityBuyTaxBps + _marketingBuyTaxBps <= 6500,
    "Buy tax cannot exceed 65%"
);
require(
    _liquiditySellTaxBps + _marketingSellTaxBps <= 6500,
    "Sell tax cannot exceed 65%"
);
// ...
}

```

If a variable denominator must be supported, the cap checks should be expressed relative to `_taxDenominator`, and the total tax should always be required to remain less than or equal to that denominator.

Resolution:

Fixed in `c20e980` :

In **KnoxNet**, the previously mutable `taxDenominator` has been replaced with the fixed constant `BPS_DENOMINATOR = 10000`, and the `_taxDenominator` parameter has been removed from `setTaxConfig`. The tax cap checks in `setTaxConfig` are now evaluated against the same denominator used by `_applyTax`, eliminating the prior mismatch between validation and effective tax calculation.

```

uint256 constant BPS_DENOMINATOR = 10000;
uint256 public constant taxDenominator = BPS_DENOMINATOR;

function setTaxConfig(
    uint256 _liquidityBuyTaxBps,
    uint256 _liquiditySellTaxBps,
    uint256 _marketingBuyTaxBps,
    uint256 _marketingSellTaxBps
) external onlyOwner {
    require(
        _liquidityBuyTaxBps + _marketingBuyTaxBps <= 6500,
        "Buy tax cannot exceed 65%"
    );
    require(
        _liquiditySellTaxBps + _marketingSellTaxBps <= 6500,
        "Sell tax cannot exceed 65%"
    );
    // ...
}

```

Evidences

PoC 1 — denominator == tax sum → 100 % effective tax

Reproduce:

```
function test_denominator_equals_tax_confiscates_everything() public {
    // setTaxConfig(0, 0, 6500, 6500, 6500) passes both require checks
    // because 6500 <= 6500, but effective tax = 6500 / 6500 = 100 %.
    token.setTaxConfig(0, 0, 6500, 6500, 6500);

    uint256 sendAmount = 1_000 * 1e18;
    uint256 contractBefore = token.balanceOf(address(token));

    vm.prank(user);
    token.transfer(recipient, sendAmount);

    uint256 received = token.balanceOf(recipient);
    uint256 contractAfter = token.balanceOf(address(token));

    // Recipient receives nothing – 100 % taken as tax.
    assertEq(received, 0, "recipient should get 0 (100% tax)");
    assertEq(
        contractAfter - contractBefore,
        sendAmount,
        "contract should absorb entire amount"
    );
}
```

Results:

```
[PASS] test_denominator_equals_tax_confiscates_everything() (gas: 126855)
```

PoC 2 — denominator < tax sum → arithmetic underflow

Reproduce:

```
function test_denominator_below_tax_reverts_all_transfers() public {
    // denominator = 5000, sellTax = 6500 → effective rate = 130 %
    // _applyTax computes: amount - (amount * 6500 / 5000) → underflow
    token.setTaxConfig(0, 0, 6500, 6500, 5000);

    uint256 sendAmount = 1_000 * 1e18;
```

```

    vm.prank(user);
    vm.expectRevert(); // arithmetic underflow in `amount - taxAmount`
    token.transfer(recipient, sendAmount);
}

```

Results:

```
[PASS] test_denominator_below_tax_reverts_all_transfers() (gas: 109940)
```

PoC 3 — same hardcoded cap, three different denominators

Reproduce:

```

function test_same_cap_different_denominators_different_rates() public {
    uint256 sendAmount = 10_000 * 1e18;
    uint256 received;

    // --- Case A: denominator 10000 → 6500/10000 = 65 % -----
    token.setTaxConfig(0, 0, 6500, 6500, 10000);

    vm.prank(user);
    token.transfer(recipient, sendAmount);
    received = token.balanceOf(recipient);

    uint256 effectiveA = ((sendAmount - received) * 10000) / sendAmount;
    assertEq(effectiveA, 6500, "Case A: effective tax should be 65%");
    emit log_named_uint("Case A effective bps", effectiveA);

    // reset recipient balance
    vm.prank(recipient);
    token.transfer(address(0xdead), received);

    // --- Case B: denominator 7000 → 6500/7000 ≈ 92.8 % -----
    token.setTaxConfig(0, 0, 6500, 6500, 7000);

    vm.prank(user);
    token.transfer(recipient, sendAmount);
    received = token.balanceOf(recipient);
}

```



```

uint256 effectiveB = ((sendAmount - received) * 10000) / sendAmount;
assertApproxEqAbs(effectiveB, 9285, 1, "Case B: ~92.85% tax");
emit log_named_uint("Case B effective bps", effectiveB);

vm.prank(recipient);
token.transfer(address(0xdead), received);

// --- Case C: denominator 6500 → 6500/6500 = 100 % -----
token.setTaxConfig(0, 0, 6500, 6500, 6500);

vm.prank(user);
token.transfer(recipient, sendAmount);
received = token.balanceOf(recipient);

uint256 effectiveC = ((sendAmount - received) * 10000) / sendAmount;
assertEq(effectiveC, 10000, "Case C: effective tax should be 100%");
}

```

Results:

```
[PASS] test_same_cap_different_denominators_different_rates() (gas: 202542)
```

[F-2026-15970](#) - Missing Transfer Event for Taxed Amount Breaks ERC20 Compliance and Causes Balance Tracking Mismatches - Medium

Description:

In **KnoxNet**, taxed transfers move a portion of tokens from the sender to the contract balance without emitting a corresponding `Transfer` event for that amount. Only the net amount sent to the recipient is emitted. This breaks the expected ERC20 event model and causes on-chain event history to diverge from actual balance changes.

In **KnoxNet**, the `transfer` and `transferFrom` flows route through `_transferFrom`, which deducts the full amount from the sender, applies tax through `_applyTax`, credits the contract with the taxed portion, and emits only a single `Transfer` event for the post-tax amount:

```
function _transferFrom(
    address sender,
    address recipient,
    uint256 amount
) internal returns (bool) {
    // ...
    _balances[sender] = _balances[sender] - amount;
    uint256 amountReceived = amount;
    if (_shouldApplyTax(sender, recipient)) {
        amountReceived = _applyTax(recipient, amount);
    }
    _balances[recipient] = _balances[recipient] + amountReceived;
    emit Transfer(sender, recipient, amountReceived);
    return true;
}
```

The taxed amount is added to the contract balance in `_applyTax`, but no `Transfer(sender, address(this), taxAmount)` event is emitted for that balance movement:

```
function _applyTax(
    address recipient,
    uint256 amount
) internal returns (uint256) {
    bool selling = liquidityPools[recipient];
    uint256 taxAmount = (amount * getTotalTaxBps(selling)) / taxDenominator;
    _balances[address(this)] += taxAmount;
}
```

```
return amount - taxAmount;
}
```

As a result, event consumers observe only the net transfer, while the contract balance increases without a matching ERC20 transfer log.

Block explorers, indexers, accounting systems, and integrations that reconstruct balances from **Transfer** events may report incorrect token movements. Tax collection becomes non-transparent at the event layer, and emitted transfer history no longer fully reflects actual token balance transitions.

Assets:

- contracts/KnoxNet.sol [<https://github.com/Knoxnetofficial/knoxnet-contracts>]

Status:

Fixed

Classification

Impact Rate:	3/5
Likelihood Rate:	3/5
Exploitability:	Independent
Complexity:	Simple
Severity:	Medium

Recommendations

Remediation:

A separate **Transfer** event should be emitted for the taxed portion whenever tokens are credited to the contract. The recipient transfer event should continue to reflect only the net amount received.

```
function _transferFrom(
    address sender,
    address recipient,
    uint256 amount
) internal returns (bool) {
    // ...
    _balances[sender] -= amount;
    uint256 amountReceived = amount;
    uint256 taxAmount;
    if (_shouldApplyTax(sender, recipient)) {
        bool selling = liquidityPools[recipient];
        taxAmount = (amount * getTotalTaxBps(selling)) / taxDenominator;
    }
}
```

```

        _balances[address(this)] += taxAmount;
        amountReceived = amount - taxAmount;
        emit Transfer(sender, address(this), taxAmount);
    }
    _balances[recipient] += amountReceived;
    emit Transfer(sender, recipient, amountReceived);
    return true;
}

```

Resolution:

Fixed in [c20e980](#) :

In **KnoxNet**, taxed transfers now emit a dedicated `Transfer` event for the portion credited to the contract. The `_applyTax` function adds the taxed amount to `address(this)` and emits `Transfer(sender, address(this), taxAmount)`, while `_transferFrom` continues to emit the net transfer to the recipient. The event stream now reflects both balance movements and aligns with ERC20 event expectations for taxed transfers.

```

function _applyTax(
    address sender,
    address recipient,
    uint256 amount
) internal returns (uint256) {
    bool selling = _classifyTransfer(sender, recipient) == TransferType.Sell;
    uint256 totalTaxBps = selling ? totalSellTaxBps : totalBuyTaxBps;
    if (totalTaxBps == 0) return amount;
    uint256 taxAmount = (amount * totalTaxBps) / taxDenominator;
    if (taxAmount == 0) return amount;
    // ...
    _balances[address(this)] += taxAmount;
    emit Transfer(sender, address(this), taxAmount);
    return amount - taxAmount;
}

```

[F-2026-15974](#) - Blacklist Enforcement Bypassed When Recipient Is Transaction-Limit Exempt - Medium

Description:

In **KnoxNet**, the blacklist check resides inside `_enforceTxLimit` after an early-return guard that exits when either party is transaction-limit exempt. If the recipient holds `isTxLimitExempt` status, the function returns before the `blacklist[sender]` check is evaluated. A blacklisted sender can therefore transfer tokens to any tx-limit-exempt address without restriction.

In **KnoxNet**, the `_enforceTxLimit` function combines two logically distinct concerns, transaction size limits and blacklist enforcement, under a single exemption guard:

```
function _enforceTxLimit(
    address sender,
    address recipient,
    uint256 amount
) internal view {
    if (isTxLimitExempt[sender] || isTxLimitExempt[recipient]) return;
    require(
        amount <= (liquidityPools[sender] ? _maxBuyTxAmount : _maxSellTxAmount
    ),
        "Amount exceeds the tx limit."
    );
    require(blacklist[sender] == 0, "Wallet blacklisted!");
}
```

The early return on the first line exits unconditionally when either `sender` or `recipient` is exempt. The blacklist check on the last line is therefore never reached. Several addresses are marked as tx-limit exempt during construction:

```
isTxLimitExempt[address(this)] = true;
isTxLimitExempt[owner()] = true;
isTxLimitExempt[routerAddress] = true;
isTxLimitExempt[DEAD] = true;
```

Any additional address granted exemption via `setIsTxLimitExempt` becomes an unrestricted transfer target for blacklisted senders as well.

Blacklisted addresses can transfer tokens to the contract, the owner, the router, the dead address, and any future tx-limit-exempt address. This partially defeats the blacklist mechanism. If a DEX pool or user wallet is later granted tx-limit exemption, blacklisted addresses can freely transfer tokens to it.

Assets:

- contracts/KnoxNet.sol [<https://github.com/Knoxnetofficial/knoxnet-contracts>]

Status:

Fixed

Classification

Impact Rate:	5/5
Likelihood Rate:	2/5
Exploitability:	Semi-Dependent
Complexity:	Simple
Severity:	Medium

Recommendations

Remediation: Blacklist enforcement should be decoupled from the transaction-limit exemption check. The blacklist validation should execute unconditionally before the early return.

Resolution: Fixed in `c20e980`:

In **KnoxNet**, blacklist enforcement has been moved ahead of the transaction-limit exemption guard within `_enforceTxLimit`. The function now checks `sender`, `recipient`, and `_msgSender` against the blacklist before evaluating `isTxLimitExempt`, so tx-limit exemption no longer bypasses blacklist restrictions. The previously described path allowing blacklisted senders to transfer to exempt addresses is therefore removed.

```
function _enforceTxLimit(
    address sender,
    address recipient,
    uint256 amount
) internal view {
    require(blacklist[sender] == 0, "Sender blacklisted!");
    require(blacklist[recipient] == 0, "Recipient blacklisted!");
    require(blacklist[_msgSender()] == 0, "Caller blacklisted!");
    if (isTxLimitExempt[sender] || isTxLimitExempt[recipient]) return;
    TransferType transferType = _classifyTransfer(sender, recipient);
    uint256 maxTxAmount = transferType == TransferType.Sell
        ? _maxSellTxAmount
        : _maxBuyTxAmount;
```

```
require(amount <= maxTxAmount, "Amount exceeds the tx limit.");  
}
```

[F-2026-15975](#) - transferOwnership Does Not Update Privileged Exemptions and Router Allowances After Ownership Transfer - Medium

Description:

In **KnoxNet**, ownership transfer updates only the stored owner address and does not migrate owner-specific privileges to the new owner or revoke them from the previous one. As a result, the previous owner retains tax, transaction-limit, and liquidity-creator exemptions, while the new owner does not automatically receive the same operational privileges. The router allowance granted to the previous owner also remains unchanged after the transfer.

In **KnoxNet**, the constructor assigns several owner-specific privileges and approvals:

```
_allowances[owner()][routerAddress] = type(uint256).max;
isTaxExempt[owner()] = true;
isLiquidityCreator[owner()] = true;
isTxLimitExempt[owner()] = true;
```

However, ownership transfer only updates `_owner`:

```
function transferOwnership(address newOwner) public virtual onlyOwner {
    require(newOwner != address(0), "Ownable: new owner is the zero address");
    _transferOwnership(newOwner);
}

function _transferOwnership(address newOwner) internal virtual {
    address oldOwner = _owner;
    _owner = newOwner;
    emit OwnershipTransferred(oldOwner, newOwner);
}
```

No state migration is performed for `isTaxExempt`, `isLiquidityCreator`, `isTxLimitExempt`, or `_allowances[owner()][routerAddress]`. The previous owner therefore remains privileged under these mappings, and the new owner must be configured manually through separate transactions.

The effective privilege model diverges from the recorded ownership state after `transferOwnership`. The previous owner may retain special transfer behavior and liquidity-related privileges, while the new owner may be unable to operate under the same assumptions until additional manual configuration is performed.

Assets:

- contracts/KnoxNet.sol [<https://github.com/Knoxnetofficial/knoxnet-contracts>]

Status:

Fixed

Classification**Impact Rate:** 4/5**Likelihood Rate:** 3/5**Exploitability:** Semi-Dependent**Complexity:** Simple**Severity:** Medium**Recommendations**

Remediation: Owner-specific exemptions and approvals should be migrated during ownership transfer. The old owner's privileges should be revoked, and the new owner should receive the required exemptions and router allowance atomically.

Resolution: Fixed in [c20e980](#):

In **KnoxNet**, ownership transfer now migrates owner-specific privileges and approvals atomically. The overridden `_transferOwnership` function revokes tax, liquidity-creator, transaction-limit, and wallet-limit exemptions from the previous owner, clears the previous owner's router allowance, and assigns the same privileges and approval to the new owner. The prior mismatch between recorded ownership and effective privileged state is therefore resolved.

```
function _transferOwnership(address newOwner) internal override {
    address oldOwner = owner();
    if (oldOwner != address(0)) {
        isTaxExempt[oldOwner] = false;
        isLiquidityCreator[oldOwner] = false;
        isTxLimitExempt[oldOwner] = false;
        isWalletLimitExempt[oldOwner] = false;
        _allowances[oldOwner][routerAddress] = 0;
    }
    super._transferOwnership(newOwner);
    if (newOwner != address(0)) {
        isTaxExempt[newOwner] = true;
        isLiquidityCreator[newOwner] = true;
    }
}
```

```
isTxLimitExempt[newOwner] = true;
isWalletLimitExempt[newOwner] = true;
_allowances[newOwner][routerAddress] = type(uint256).max;
    }
}
```

[F-2026-15980](#) - Fee Distribution in `_autoSwapBack` Uses Configured Tax Rates as Proxy for Actual Fee Composition, Causing Systematic Misallocation Between Marketing and Liquidity - Medium

Description:

In **KnoxNet**, all fee tokens from both buys and sells are collected into a single undifferentiated balance. When distributing, `_autoSwapBack` assumes that the marketing/liquidity composition of this balance matches the ratio of configured tax rates. In practice, the actual composition depends on real trading volumes, which the contract does not track. When buy and sell tax structures differ, the distribution formula systematically over- or under-allocates to one receiver at the expense of the other.

In **KnoxNet**, `_applyTax` collects all fee tokens into one shared pool regardless of whether the fee originated from a buy or a sell, and regardless of the marketing/liquidity split configured for that direction:

```
_balances[address(this)] += taxAmount;
```

When `_autoSwapBack` distributes this pool, it calculates the liquidity share using the ratio of configured rates:

```
uint256 combinedTaxBps = totalBuyTaxBps + totalSellTaxBps;
uint256 totalLiquidityTaxBps = liquidityBuyTaxBps + liquiditySellTaxBps;
uint256 amountToLiquify = ((amountToSwap * totalLiquidityTaxBps) / 2) /
    combinedTaxBps;
```

This formula produces the correct result only when buy volume and sell volume contribute equally to the pool. Consider a configuration where buy tax is entirely liquidity-oriented and sell tax is entirely marketing-oriented:

```
liquidityBuyTaxBps = 500,   marketingBuyTaxBps = 0    → totalBuy = 500
liquiditySellTaxBps = 0,    marketingSellTaxBps = 6500 → totalSell = 6500
```

The formula computes liquidity share as `500 / 7000 = 7.1%` regardless of trading activity. But real fee composition depends on volumes:

```
Scenario A – mostly sells:
Buys:  1M tokens × 5% = 50K tokens (100% liquidity)
Sells: 4M tokens × 65% = 2.6M tokens (100% marketing)
Pool total: 2.65M tokens
Real liquidity share:  50K / 2.65M = 1.9%
Formula applies:      7.1%
Result: liquidity receives ~3.7x more than it should
        (taken from marketing's share)
```

Scenario B – mostly buys:

Buys: 4M tokens × 5% = 200K tokens (100% liquidity)

Sells: 1M tokens × 65% = 650K tokens (100% marketing)

Pool total: 850K tokens

Real liquidity share: 200K / 850K = 23.5%

Formula applies: 7.1%

Result: liquidity receives ~3.3x less than it should
(surplus goes to marketing)

The same 7.1% ratio propagates through all three allocation stages: token reservation for LP, ETH denominator calculation, and ETH split. The error direction is consistent across all stages and compounds in the same direction.

Fee distribution between marketing and liquidity receivers may deviate significantly from the fees actually collected for each purpose. The magnitude scales with the asymmetry between buy-side and sell-side tax configurations and with the imbalance between buy and sell trading volumes. User balances are not affected. Both receivers are currently configured to the same owner-controlled address, which limits practical impact under the current deployment configuration.

Assets:

- contracts/KnoxNet.sol [<https://github.com/Knoxnetofficial/knoxnet-contracts>]

Status:

Fixed

Classification

Impact Rate: 3/5

Likelihood Rate: 4/5

Exploitability: Independent

Complexity: Simple

Severity: Medium

Recommendations

Remediation:

Fee accrual should be tracked separately by purpose instead of aggregating all taxed tokens into a single undifferentiated balance. Distinct counters should be maintained for marketing and liquidity allocations, and `_autoSwapBack` should consume those tracked amounts directly.

If the current approximation is intentional, the resulting allocation inaccuracy should be documented explicitly.

Resolution:

Fixed in [c20e980](#) :

In **KnoxNet**, fee accrual has been replaced with per-purpose tracking using dedicated `accumulatedMarketingTokens` and `accumulatedLiquidityTokens` counters. The `_applyTax` function now splits each taxed amount at the point of collection based on the actual buy/sell direction and its configured tax composition, rather than deferring the split to distribution time. The `_autoSwapBack` function consumes these tracked amounts directly and distributes proportionally to the real accumulated balances, eliminating the prior dependency on a static rate-based formula that ignored actual trading volumes.

```
function _applyTax(
    address sender,
    address recipient,
    uint256 amount
) internal returns (uint256) {
    bool selling = _classifyTransfer(sender, recipient) == TransferType.Sell;
    uint256 totalTaxBps = selling ? totalSellTaxBps : totalBuyTaxBps;
    // ...
    uint256 liquidityTaxBps = selling ? liquiditySellTaxBps : liquidityBuyTaxBps;
    uint256 liquidityTokens = (taxAmount * liquidityTaxBps) / totalTaxBps;
    uint256 marketingTokens = taxAmount - liquidityTokens;
    accumulatedLiquidityTokens += liquidityTokens;
    accumulatedMarketingTokens += marketingTokens;
    // ...
}

function _autoSwapBack(uint256 amount) internal swapping {
    // ...
    uint256 totalAccumulatedTokens = accumulatedMarketingTokens + accumulatedLiquidityTokens;
    if (totalAccumulatedTokens == 0) return;
    if (amountToSwap > totalAccumulatedTokens) amountToSwap = totalAccumulatedTokens;
    uint256 liquidityPortion = (accumulatedLiquidityTokens * amountToSwap) / totalAccumulatedTokens;
    uint256 marketingPortion = amountToSwap - liquidityPortion;
    // ...
}
```

[F-2026-15984](#) - Incomplete Transfer Classification Causes Inconsistent Limit and Tax Behavior for Wallet-to-Wallet Transfers - Medium

Description:

In **KnoxNet**, transfer operations are classified using binary conditions that distinguish only between "buy" and "not buy" or "sell" and "not sell." No dedicated classification exists for wallet-to-wallet transfers. As a result, the same wallet-to-wallet operation is simultaneously subject to sell-side transaction limits and buy-side tax rates, producing internally contradictory behavior across the limit and tax subsystems.

In **KnoxNet**, transaction-limit enforcement in `_enforceTxLimit` classifies transfers based solely on whether `sender` is a liquidity pool:

```
require(
    amount <= (liquidityPools[sender] ? _maxBuyTxAmount : _maxSellTxAmount),
    "Amount exceeds the tx limit."
);
```

When `sender` is not a pool, the transfer is evaluated against `_maxSellTxAmount` regardless of whether `recipient` is a pool. Wallet-to-wallet transfers therefore inherit the sell-side limit.

Separately, the tax subsystem in `_applyTax` classifies transfers based on whether `recipient` is a liquidity pool:

```
function _applyTax(
    address recipient,
    uint256 amount
) internal returns (uint256) {
    bool selling = liquidityPools[recipient];
    uint256 taxAmount = (amount * getTotalTaxBps(selling)) / taxDenominator;
    // ...
}
```

When `recipient` is not a pool, the transfer is taxed at the buy rate. Wallet-to-wallet transfers therefore inherit the buy-side tax.

This means the same wallet-to-wallet operation is treated as a sell by the limit system and as a buy by the tax system. These two classifications are mutually inconsistent and neither correctly represents a regular transfer. The contract lacks a third transfer category that would apply dedicated limits and tax behavior to non-DEX token movements.

Under default configuration both limits are equal (1%) and buy tax is lower than sell tax, so the inconsistency produces a combination of the stricter limit with the softer tax. If the owner later configures divergent buy and sell limits, wallet-to-wallet transfers become restricted by the sell-side cap even though no pool interaction occurs. Conversely, when `transferTax` is enabled, such transfers are taxed at the buy rate, which is typically significantly lower than the sell rate.

Wallet-to-wallet transfers may be blocked by a sell limit that does not conceptually apply to them, creating unexpected restrictions whenever buy and sell transaction limits differ. When `transferTax` is enabled, the same transfers are taxed at the buy rate rather than at a rate appropriate for non-DEX movements. This dual misclassification produces internally contradictory transfer behavior and may confuse both users and integrators about the effective rules governing regular token movements.

Assets:

- `contracts/KnoxNet.sol` [<https://github.com/Knoxnetofficial/knoxnet-contracts>]

Status:

Mitigated

Classification

Impact Rate: 3/5

Likelihood Rate: 4/5

Exploitability: Independent

Complexity: Simple

Severity: Medium

Recommendations

Remediation: A three-way transfer classification should be introduced so that buy, sell, and regular transfer operations are identified and handled distinctly. Both the limit and tax subsystems should use the same classification logic.

Resolution: Mitigated in `42b580b` :

In **KnoxNet**, wallet-to-wallet transfers are now classified through the shared `TransferType` flow, and both `_enforceTxLimit` and `_applyTax` apply the same interpretation. The prior inconsistency, under which wallet-to-wallet transfers were treated as sells for limits and buys for taxation, has therefore been removed.

The underlying recommendation has not been implemented in full.

`TransferType.Transfer` still does not have dedicated limit and tax parameters.

The latest commit explicitly documents that wallet-to-wallet transfers are intended to use buy-side treatment in both subsystems, which clarifies the design choice but does not introduce a distinct transfer-specific policy.

[F-2026-16037](#) - Auto-Swap Reverts During Sell Execution Can Deny User Exits - Medium

Description:

In **KnoxNet**, the `_transferFrom` function invokes `_autoSwapBack` during qualifying sell transactions, coupling the user's sell path to external Uniswap router calls. If the internal tax swap or liquidity addition reverts, the entire user sell transaction reverts as well. As a result, users can be prevented from selling whenever the auto-swap path becomes temporarily invalid or fails at execution time.

In **KnoxNet**, the `_transferFrom` function triggers `_autoSwapBack` synchronously inside the taxed transfer flow:

```
if (_shouldApplyTax(sender, recipient)) {
    amountReceived = _applyTax(recipient, amount);
    if (_shouldAutoSwap(recipient) && amount > 0) _autoSwapBack(amount);
}
```

The `_autoSwapBack` function then performs external router operations that are not isolated from the user transaction. Both the token-to-ETH swap and the optional liquidity addition are executed inline, so any revert in either call propagates back to the original sell:

```
router.swapExactTokensForETHSupportingFeeOnTransferTokens(
    amountToSwap,
    0,
    path,
    address(this),
    block.timestamp
);
// ...
if (amountToLiquify > 0) {
    router.addLiquidityETH{value: amountETHLiquidity}(
        address(this),
        amountToLiquify,
        0,
        0,
        liquidityTaxReceiver,
        block.timestamp
    );
}
```

This design creates multiple revert paths. For example, rounding can reduce the swappable amount to zero after the liquidity split, or liquidity addition can be attempted with an invalid ETH/token ratio. Because these

failures occur inside the sell path, the user transaction cannot complete independently of the tax-processing logic.

User sell transactions can revert even when the transfer itself is otherwise valid. This can temporarily block exits for all affected sellers until auto-swap conditions change or privileged configuration is updated.

Assets:

- contracts/KnoxNet.sol [<https://github.com/Knoxnetofficial/knoxnet-contracts>]

Status:

Fixed

Classification

Impact Rate: 3/5

Likelihood Rate: 3/5

Exploitability: Independent

Complexity: Simple

Severity: Medium

Recommendations

Remediation:

The tax-processing path should be decoupled from the user sell path. External router calls inside `_autoSwapBack` should be wrapped in `try/catch`, or failed auto-swap attempts should be skipped and deferred so that the sell transaction can still complete.

Resolution:

Fixed in `c20e980`:

In **KnoxNet**, the auto-swap path has been isolated from the user sell flow by wrapping the external router calls in `try/catch` and returning on failure instead of propagating a revert. The `swapExactTokensForETHSupportingFeeOnTransferTokens` call and the optional `addLiquidityETH` call no longer cause the enclosing sell transaction to fail when the auto-swap path is temporarily invalid.

```
if (_shouldApplyTax(sender, recipient)) {
    amountReceived = _applyTax(sender, recipient, amount);
    if (_shouldAutoSwap(recipient)) _autoSwapBack(amount);
}
function _autoSwapBack(uint256 amount) internal swapping {
    // ...
    try
```

```

        router.swapExactTokensForETHSupportingFeeOnTransferTokens(
            swapTokens,
            0,
            path,
            address(this),
            block.timestamp + ROUTER_DEADLINE_SECONDS
        )
    } catch {
        return;
    }
    // ...
    try
        router.addLiquidityETH(value: amountETHLiquidity){
            address(this),
            liquidityTokensToLiquify,
            minToken,
            minETH,
            liquidityTaxReceiver,
            block.timestamp + ROUTER_DEADLINE_SECONDS
        }
    returns (uint256, uint256 ethUsed, uint256) {
        accumulatedLiquidityTokens -= liquidityTokensToLiquify;
        addedLiquidityTokens = liquidityTokensToLiquify;
        addedLiquidityETH = ethUsed;
    } catch {
        // skip liquidity addition on failure; tokens + ETH remain in contract
    }
}

```

[F-2026-15971](#) - Zero Slippage Protection in Swap Functions Exposes Tax Conversions to Sandwich Attacks - Low

Description:

In **KnoxNet**, all token-to-ETH swap operations are executed with `amountOutMin` set to `0`. This allows swaps to proceed regardless of how little ETH is returned at execution time. As a result, tax conversions can be front-run and back-run by external traders, causing the contract to realize materially worse execution than expected.

In **KnoxNet**, both the automatic tax conversion path in `_autoSwapBack` and the manual recovery path in `manualClogClearPercent` submit swaps without any slippage bound:

```
router.swapExactTokensForETHSupportingFeeOnTransferTokens(
    amountToSwap,
    0,
    path,
    address(this),
    block.timestamp
);
```

The same pattern is used in `manualClogClearPercent`:

```
router.swapExactTokensForETHSupportingFeeOnTransferTokens(
    amountToSwap,
    0,
    path,
    address(this),
    block.timestamp
);
```

Because no minimum output is enforced, these swaps remain valid even if the pool price is temporarily manipulated immediately before execution. This permits third parties to sandwich the transaction and extract value from the contract's swap flow, while the contract accepts the degraded output unconditionally.

The contract may receive substantially less ETH than the prevailing market price for the same token amount. This reduces the value delivered to the configured tax receivers and can materially worsen execution during large swap operations. Repeated exploitation can create persistent loss during tax conversion.

Assets:

- contracts/KnoxNet.sol [<https://github.com/Knoxnetofficial/knoxnet-contracts>]

Status:

Accepted

Classification**Impact Rate:** 3/5**Likelihood Rate:** 2/5**Exploitability:** Independent**Complexity:** Simple**Severity:** Low**Recommendations****Remediation:**

A minimum acceptable ETH output should be enforced for each swap instead of using `0` for `amountOutMin`. The value may be derived from router quotes with a bounded slippage tolerance, or from an oracle-backed pricing check.

```
uint256[] memory amounts = router.getAmountsOut(amountToSwap, path);
uint256 minOut = (amounts[1] * 95) / 100;
router.swapExactTokensForETHSupportingFeeOnTransferTokens(
    amountToSwap,
    minOut,
    path,
    address(this),
    block.timestamp
);
```

If router quoting is adopted, the router interface should be extended accordingly and the slippage tolerance should be configurable within a bounded range.

Resolution:**Accepted:**

In **KnoxNet**, token-to-ETH swaps in both the automatic tax-conversion flow and the manual clearing flow still use `amountOutMin = 0`. No minimum-output protection has been introduced, and the current implementation explicitly documents this behavior as intentional. The issue therefore remains unresolved and has been accepted as a design decision.

F-2026-15972 - Incorrect blacklistCount Accounting on Duplicate Entries Causes Persistent State Inconsistency - Low

Description:

In **KnoxNet**, the `blacklistCount` value is updated mechanically for each array element processed in `updateBlacklist`, without checking whether the address is already in the requested state. If the same address appears multiple times or is blacklisted more than once across calls, the counter is incremented repeatedly for a single logical blacklist entry. This causes `blacklistCount` to diverge from the actual number of blacklisted addresses.

In **KnoxNet**, the `updateBlacklist` function adjusts `blacklistCount` before updating the stored blacklist state:

```
function updateBlacklist(
    address[] calldata _wallets,
    bool _blacklist
) external onlyTrustedOperator {
    for (uint i = 0; i < _wallets.length; i++) {
        if (_blacklist) {
            blacklistCount++;
            emit WalletRestricted(tx.origin, _wallets[i], block.number);
        } else {
            if (blacklist[_wallets[i]] != 0) blacklistCount--;
        }
        blacklist[_wallets[i]] = _blacklist ? block.number : 0;
    }
}
```

When `_blacklist` is `true`, the counter is incremented unconditionally, even if `blacklist[_wallets[i]]` is already non-zero. As a result, repeated submissions of the same address inflate the counter. If duplicates are present in the same input array, the first unblacklist iteration clears the entry and decrements once, while later duplicate iterations do not decrement again because the stored value is already zero. The counter can therefore remain permanently overstated.

`blacklistCount` cannot be relied upon as an accurate representation of the number of blacklisted addresses. Any monitoring, administrative reporting, or off-chain logic that depends on this value may operate on incorrect state. Once inflated through duplicate processing, the counter may remain inconsistent indefinitely.

Assets:

- `contracts/KnoxNet.sol` [<https://github.com/Knoxnetofficial/knoxnet-contracts>]

Status:

Fixed

Classification

Impact Rate: 2/5

Likelihood Rate: 3/5

Exploitability: Independent

Complexity: Simple

Severity: Low

Recommendations

Remediation: `blacklistCount` should be updated only when an address actually changes state. The previous status should be checked before incrementing or decrementing the counter.

Resolution: Fixed in `c20e980` :

In **KnoxNet**, `blacklistCount` is now updated only when an address actually changes blacklist state. The `updateBlacklist` function first determines whether each address is already blacklisted and increments or decrements the counter only on a real transition. This prevents duplicate entries or repeated blacklist operations from inflating the counter and restores `blacklistCount` as an accurate count of blacklisted addresses.

```
function updateBlacklist(
  address[] calldata _wallets,
  bool _blacklist
) external onlyTrustedOperator {
  for (uint i = 0; i < _wallets.length; i++) {
    bool isCurrentlyBlacklisted = blacklist[_wallets[i]] != 0;
    if (_blacklist) {
      if (!isCurrentlyBlacklisted) {
        blacklistCount++;
        emit WalletRestricted(_msgSender(), _wallets[i], block.number);
      }

      blacklist[_wallets[i]] = block.number;
    } else {
      if (isCurrentlyBlacklisted) {
        blacklistCount--;
        blacklist[_wallets[i]] = 0;
      }
    }
  }
}
```

}

}

[F-2026-15973](#) - airdrop Bypasses Transfer Restrictions and Allows Unrestricted Token Distribution - Low

Description:

In **KnoxNet**, the `airdrop` function performs token transfers through `_basicTransfer` instead of the standard transfer path used by `transfer` and `transferFrom`. As a result, the distributed tokens are moved without applying blacklist checks, tax logic, trading-status restrictions, transaction limits, or wallet limits. The administrative distribution path is therefore not subject to the same controls enforced for regular token transfers.

In **KnoxNet**, the `airdrop` function loops over recipients and directly invokes `_basicTransfer`:

```
function airdrop(
    address[] calldata addresses,
    uint256[] calldata amounts
) external onlyOwner {
    // ...
    for (uint i = 0; i < addresses.length; i++) {
        if (!liquidityPools[addresses[i]] && !isLiquidityCreator[addresses[i]]) {
            _basicTransfer(from, addresses[i], amounts[i] * (10 ** _decimals))
        }
    }
}
```

The `_basicTransfer` function only updates balances and emits a `Transfer` event:

```
function _basicTransfer(
    address sender,
    address recipient,
    uint256 amount
) internal returns (bool) {
    _balances[sender] = _balances[sender] - amount;
    _balances[recipient] = _balances[recipient] + amount;
    emit Transfer(sender, recipient, amount);
    return true;
}
```

Unlike `transferFrom`, this path does not call `_enforceTxLimit`, `_enforceWalletLimit`, `_shouldApplyTax`, or the trading launch checks. The

transfer-control model is therefore bypassed whenever tokens are distributed through `airdrop`.

Tokens may be distributed to addresses that would otherwise be blocked or constrained by the normal transfer flow. Wallet size limits, blacklist enforcement, taxation, and pre-launch transfer restrictions are not applied to airdropped transfers. This creates an unrestricted privileged transfer path that is inconsistent with the contract's standard token controls.

Assets:

- `contracts/KnoxNet.sol` [<https://github.com/Knoxnetofficial/knoxnet-contracts>]

Status:

Mitigated

Classification

Impact Rate: 3/5

Likelihood Rate: 2/5

Exploitability: Dependent

Complexity: Simple

Severity: Low

Recommendations

Remediation:

- The `airdrop` function should route distributions through the same transfer logic used for standard token movements, or explicitly enforce the same checks before calling `_basicTransfer`.
- If selected restrictions are intended to be bypassed for administrative distributions, those exceptions should be enforced explicitly and documented instead of relying on `_basicTransfer`.

Resolution:

Mitigated:

In **KnoxNet**, the `airdrop` function still routes distributions through `_basicTransfer` rather than the standard `_transferFrom` path. The current implementation explicitly documents this behavior as intentional.

```
// Administrative distribution helper.  
// Intentionally bypasses tax, trading-status restrictions, tx limits, wallet  
// limits, and blacklist checks  
// by routing through _basicTransfer (owner-only function).  
function airdrop(  

```

```

    address[] calldata addresses,
    uint256[] calldata amounts
) external onlyOwner {
    require(addresses.length > 0 && amounts.length == addresses.length);
    address from = _msgSender();
    for (uint i = 0; i < addresses.length; i++) {
        require(addresses[i] != address(0), "Airdrop to zero address");
        if (!liquidityPools[addresses[i]] && !isLiquidityCreator[addresses[i]]
    ) {
        _basicTransfer(from, addresses[i], amounts[i] * (10 ** _decimals))
    ;
    }
    }
}

```

```


```

F-2026-15977 - Lack of Timelock on Critical Admin Functions Enables Immediate High-Impact Configuration Changes - Low

Description:

In **KnoxNet**, multiple privileged functions can be executed immediately by the owner or trusted operators without any timelock, delay, or queued execution model. This allows high-impact parameter changes to take effect within a single transaction. As a result, users and integrators are given no opportunity to observe and react before sensitive configuration changes are applied.

In **KnoxNet**, critical administrative functions are protected only by `onlyOwner` or `onlyTrustedOperator`, and no delayed execution mechanism is present. Examples include tax updates, blacklist updates, transaction-limit changes, tax-receiver changes, and ETH recovery:

```
function setTaxConfig(...) external onlyOwner { /* ... */ }
function setTxLimit(...) external onlyOwner { /* ... */ }
function setMaxWallet(...) external onlyOwner { /* ... */ }
function setTaxReceivers(...) external onlyOwner { /* ... */ }
function updateBlacklist(...) external onlyTrustedOperator { /* ... */ }
function recoverStuckETH(...) external onlyTrustedOperator { /* ... */ }
function openTrading() external onlyTrustedOperator { /* ... */ }
```

These functions directly mutate live protocol state when called. No delay, announcement period, or acceptance window is enforced before changes become effective. The absence of a timelock is particularly relevant because these functions govern taxation, transfer restrictions, blacklist behavior, operational fund flows, and launch status.

Critical token parameters may be changed without advance notice, and the new configuration becomes effective immediately. Users may be exposed to abrupt trading restrictions, blacklist actions, or tax changes with no time to adjust their positions or integrations. This increases operational and governance risk around privileged administration.

Assets:

- `contracts/KnoxNet.sol` [<https://github.com/Knoxnetofficial/knoxnet-contracts>]

Status:

Accepted

Classification

Impact Rate:

4/5

Likelihood Rate:	3/5
Exploitability:	Dependent
Complexity:	Simple
Severity:	Low

Recommendations

Remediation: A timelock should be introduced for high-impact administrative actions. Sensitive state changes should be queued first and executed only after a mandatory delay.

Resolution: **Accepted:**

In **KnoxNet**, high-impact administrative functions remain executable immediately under `onlyOwner` or `onlyTrustedOperator`, and no timelock, delay, or queued execution mechanism has been introduced. The current implementation explicitly documents that immediate execution without a timelock is intentional. The issue therefore remains unresolved and has been accepted as a design decision.

```

/// @notice Administrative functions (tax config, limits, blacklist, etc.) execute immediately
/// without a timelock by design. Operational security relies on owner/trusted-operator access
/// control and off-chain monitoring via emitted events.
function setTaxConfig(...) external onlyOwner { /* ... */ }
function setTxLimit(...) external onlyOwner { /* ... */ }
function setMaxWallet(...) external onlyOwner { /* ... */ }
function setTaxReceivers(...) external onlyOwner { /* ... */ }
function updateBlacklist(...) external onlyTrustedOperator { /* ... */ }
function recoverStuckETH(...) external onlyTrustedOperator { /* ... */ }
function openTrading() external onlyTrustedOperator { /* ... */ }

```

[F-2026-15978](#) - Wallet Limit Validation Uses Gross Transfer Amount and Can Reject Transfers That Would Satisfy the Post-Tax Limit - Low

Description:

In **KnoxNet**, the wallet-size check is performed using the full transfer amount before tax is applied. When transfer tax is enabled, the recipient receives only the net post-tax amount, but the limit check is still evaluated against the gross amount. As a result, transfers may revert even though the recipient's final balance would remain within the configured wallet limit.

In **KnoxNet**, `_transferFrom` enforces the wallet limit before tax is calculated:

```
if (!liquidityPools[recipient] && recipient != DEAD) {  
    if (!isTxLimitExempt[recipient]) {  
        _enforceWalletLimit(recipient, amount);  
    }  
}
```

The `_enforceWalletLimit` function compares the current recipient balance plus the gross `amount` against `_maxWalletSize`:

```
function _enforceWalletLimit(address recipient, uint256 amount) internal view  
{  
    uint256 walletLimit = _maxWalletSize;  
    require(  
        _balances[recipient] + amount <= walletLimit,  
        "Amount exceeds the max wallet size."  
    );  
}
```

However, the actual credited amount may be smaller because tax is applied later in `_applyTax`:

```
if (_shouldApplyTax(sender, recipient)) {  
    amountReceived = _applyTax(recipient, amount);  
}
```

The wallet-limit check is therefore performed against a value that the recipient never actually receives when taxation applies.

Transfers may be rejected even when the recipient's post-tax balance would remain below the configured wallet cap. This makes the effective wallet limit stricter than the configured value and can unnecessarily restrict

taxed transfers, including buy or wallet-to-wallet transfers when transfer tax is enabled.

Assets:

- contracts/KnoxNet.sol [<https://github.com/Knoxnetofficial/knoxnet-contracts>]

Status:

Fixed

Classification

Impact Rate:	2/5
Likelihood Rate:	3/5
Exploitability:	Independent
Complexity:	Simple
Severity:	Low

Recommendations

Remediation:

The wallet-limit check should use the net amount that will actually be credited to the recipient after taxation. The taxed amount should be determined before enforcing the wallet cap.

```
uint256 amountReceived = amount;
if (_shouldApplyTax(sender, recipient)) {
    bool selling = liquidityPools[recipient];
    uint256 taxAmount = (amount * getTotalTaxBps(selling)) / taxDenominator;
    amountReceived = amount - taxAmount;
}
if (!liquidityPools[recipient] && recipient != DEAD && !isTxLimitExempt[recipient]) {
    _enforceWalletLimit(recipient, amountReceived);
}
```

If the tax calculation remains inside `_applyTax`, the wallet-limit check should be moved after the net transfer amount is determined.

Resolution:

Fixed in `c20e980` :

In **KnoxNet**, the wallet-limit check has been moved to occur after the net transfer amount is determined. The `_transferFrom` function now computes `amountReceived`, including any tax deduction through `_applyTax`, before calling `_enforceWalletLimit`. As a result, the wallet cap is evaluated against

the actual amount credited to the recipient rather than the gross pre-tax amount.

```
_balances[sender] = _balances[sender] - amount;
uint256 amountReceived = amount;
if (_shouldApplyTax(sender, recipient)) {
    amountReceived = _applyTax(sender, recipient, amount);
    if (_shouldAutoSwap(recipient)) _autoSwapBack(amount);
}
if (!liquidityPools[recipient] && recipient != DEAD) {
    if (!isWalletLimitExempt[recipient]) {
        _enforceWalletLimit(recipient, amountReceived);
    }
}
_balances[recipient] = _balances[recipient] + amountReceived;
```


[F-2026-15981](#) - Limited On-Chain Visibility of Tax Configuration

Obscures Effective Tax State - Low

Description:

In **KnoxNet**, the underlying tax configuration variables are not exposed through public state visibility or a dedicated getter. As a result, the full active tax model cannot be verified through straightforward on-chain reads, even though these values directly determine transfer taxation. This reduces transparency around the effective tax state and compounds the impact of [F-2026-15969](#), where an unvalidated `taxDenominator` can invalidate the intended tax caps.

In **KnoxNet**, the key tax parameters are stored in non-public state variables:

```
uint256 marketingBuyTaxBps = 500;
uint256 marketingSellTaxBps = 9000;
uint256 liquidityBuyTaxBps = 0;
uint256 liquiditySellTaxBps = 0;
uint256 totalBuyTaxBps = marketingBuyTaxBps + liquidityBuyTaxBps;
uint256 totalSellTaxBps = marketingSellTaxBps + liquiditySellTaxBps;
uint256 taxDenominator = 10000;
```

Only partial visibility is provided through `getTotalTaxBps`, which exposes aggregate buy or sell tax values:

```
function getTotalTaxBps(bool selling) public view returns (uint256) {
    if (selling) return totalSellTaxBps;
    return totalBuyTaxBps;
}
```

However, this does not expose the underlying denominator or the split between marketing and liquidity components. This becomes particularly relevant in the context of [F-2026-15969](#), where the effective tax rate depends not only on the numerator values but also on the mutable `taxDenominator`. A user or integration reading only aggregate tax values may therefore form an incomplete or misleading view of the actual configured tax behavior.

Users, integrators, and monitoring systems may be unable to reliably verify the full active tax configuration from simple on-chain reads. This weakens operational transparency and makes it harder to detect or monitor the conditions described in [F-2026-15969](#). Incomplete visibility may lead external systems to rely on aggregate tax figures that do not fully describe the effective tax model.

Assets:

- contracts/KnoxNet.sol [<https://github.com/Knoxnetofficial/knoxnet-contracts>]

Status:

Fixed

Classification

Impact Rate: 2/5

Likelihood Rate: 3/5

Exploitability: Independent

Complexity: Simple

Severity: Low

Recommendations

Remediation: The full tax configuration should be exposed through public state variables or a dedicated getter that returns all tax parameters, including `taxDenominator`.

Resolution: Fixed in `c20e980`:

In **KnoxNet**, all tax configuration variables are now declared with `public` visibility, including `marketingBuyTaxBps`, `marketingSellTaxBps`, `liquidityBuyTaxBps`, `liquiditySellTaxBps`, `totalBuyTaxBps`, `totalSellTaxBps`, and `taxDenominator`. The full tax model, including the per-purpose split and the denominator, is directly readable on-chain through the auto-generated getter functions. The cross-reference with F-2026-15969 is also resolved, as `taxDenominator` is now a `public constant` fixed at `10000`.

```
uint256 public marketingBuyTaxBps = 500;
uint256 public marketingSellTaxBps = 500;
uint256 public liquidityBuyTaxBps = 0;
uint256 public liquiditySellTaxBps = 0;
uint256 public totalBuyTaxBps = marketingBuyTaxBps + liquidityBuyTaxBps;
uint256 public totalSellTaxBps = marketingSellTaxBps + liquiditySellTaxBps;
uint256 public constant taxDenominator = BPS_DENOMINATOR;
```

[F-2026-15985](#) - Non-Reentrant-Safe swapping Modifier Can Be Corrupted Through Nested Execution via Marketing Receiver Callback - Low

Description:

In **KnoxNet**, the `swapping` modifier unconditionally resets `inSwap` to `false` on exit without verifying whether it was the outermost invocation. During `_autoSwapBack`, a low-level ETH transfer to `marketingTaxReceiver` passes execution control to an external address while `inSwap` is `true`. If the receiver re-enters a function protected by the same modifier, the nested call resets `inSwap` to `false` upon completion. This corrupts the flag for the still-executing outer function and causes subsequent operations to proceed outside the intended swap-guarded context.

In **KnoxNet**, the `swapping` modifier is implemented as a simple boolean toggle:

```
modifier swapping() {  
    inSwap = true;  
    _;  
    inSwap = false;  
}
```

Unlike a standard reentrancy guard, no `require(!inSwap)` check is present. If two `swapping`-protected functions are nested, the inner call silently resets `inSwap = false` on exit.

During `_autoSwapBack`, ETH is sent to the marketing receiver between the swap and the liquidity addition:

```
(bool sentMarketing, ) = marketingTaxReceiver.call{  
    value: amountETHMarketing  
}("");  
// ...  
if (amountToLiquify > 0) {  
    router.addLiquidityETH{value: amountETHLiquidity}(  
        address(this),  
        amountToLiquify,  
        0, 0,  
        liquidityTaxReceiver,  
        block.timestamp  
    );  
}
```

If `marketingTaxReceiver` is a contract that re-enters `manualClogClearPercent` (which also uses the `swapping` modifier), the nested execution resets `inSwap = false` upon return. The outer `_autoSwapBack` then continues with `inSwap = false`, causing the subsequent `addLiquidityETH` call to route through the full `_transferFrom` path instead of `_basicTransfer`. This means the contract's own liquidity transfer is subject to taxation and may trigger recursive auto-swap execution.

```
_autoSwapBack [inSwap = true]
├─ router.swap(...) // OK: inSwap = true
├─ marketingTaxReceiver.call{value: ETH}
│   └─ re-enters manualClogClearPercent
│       └─ swapping: inSwap = true (no-op)
│           └─ ... executes ...
│               └─ swapping EXIT: inSwap = false ← CORRUPTED
├─ router.addLiquidityETH(...) // inSwap = false!
│   └─ transferFrom → _transferFrom
│       └─ inSwap == false → normal path
│           └─ tax applied on contract's own transfer
│               └─ _shouldAutoSwap may trigger recursion
└─ swapping EXIT: inSwap = false (already false)
```

After flag corruption, the contract's own liquidity addition is taxed and may trigger recursive auto-swap behavior. This can cause unexpected token loss to self-taxation, unpredictable balance changes, or transaction failure. Exploitation requires `marketingTaxReceiver` to be a contract with owner-level access, limiting the vector to owner-controlled configurations.

Assets:

- `contracts/KnoxNet.sol` [<https://github.com/Knoxnetofficial/knoxnet-contracts>]

Status:

Fixed

Classification

Impact Rate:	4/5
Likelihood Rate:	2/5
Exploitability:	Semi-Dependent
Complexity:	Medium
Severity:	Low

Recommendations

Remediation:

The `swapping` modifier should be implemented as a proper reentrancy guard that rejects nested calls instead of silently resetting the flag.

```
modifier swapping() {  
    require(!inSwap, "Reentrant swap");  
    inSwap = true;  
    _;  
    inSwap = false;  
}
```

This prevents any nested `swapping`-protected function from executing during an active swap and eliminates the flag corruption path entirely.

Resolution:

Fixed in `c20e980` :

In **KnoxNet**, the `swapping` modifier now includes a `require(!inSwap)` guard that rejects nested invocations. If a re-entrant call attempts to execute a `swapping`-protected function while `_autoSwapBack` or `manualClogClearPercent` is already in progress, the transaction reverts instead of silently resetting the flag. The flag corruption path described in the finding is therefore eliminated.

```
modifier swapping() {  
    require(!inSwap, "Reentrant swap");  
    inSwap = true;  
    _;  
    inSwap = false;  
}
```

F-2026-15996 - Implicit Infinite Router Allowance in the Constructor Grants Spending Authority Over the Owner Balance - Low

Description:

In **KnoxNet**, the constructor grants the hardcoded router an infinite allowance not only over the contract's own token balance, but also over the deployer's balance. While the contract-side approval is needed for swap and liquidity operations, the owner-side approval is not strictly required at deployment and grants immediate third-party spending authority over the full initial supply. This creates an unnecessary privileged approval and increases the impact of router misconfiguration.

In **KnoxNet**, the constructor sets the following allowances:

```
_allowances[owner()][routerAddress] = type(uint256).max;  
_allowances[address(this)][routerAddress] = type(uint256).max;
```

The second allowance allows the router to move contract-held fee tokens during `_autoSwapBack`, which is expected. However, the first allowance gives the router unrestricted `transferFrom` access to the owner's balance immediately after deployment, even though the owner could approve the router explicitly only when adding liquidity.

Because the full supply is minted to the owner in the same constructor, this approval covers the entire initial token inventory from the outset. The risk is further amplified if deployment occurs on a chain where the hardcoded `routerAddress` does not correspond to the intended trusted router, or if router migration assumptions change over time.

The router is authorized to transfer the owner's tokens without any separate approval transaction from the owner after deployment. If the configured router address is incorrect for the target chain, the full owner balance may be exposed to an unintended external contract. Even on the intended chain, the approval is broader than operationally necessary.

Assets:

- contracts/KnoxNet.sol [<https://github.com/Knoxnetofficial/knoxnet-contracts>]

Status:

Mitigated

Classification

Impact Rate: 3/5

Likelihood Rate: 2/5

Exploitability: Independent

Complexity: Simple

Severity: Low

Recommendations

Remediation: Router approvals should not be granted as standing infinite allowances, whether over the owner balance or the contract balance. The owner-side approval should be removed entirely, and router access should be granted only for the exact token amount required immediately before each swap or liquidity operation. After execution, the allowance should be reset to zero or reduced back to the remaining required amount.

Resolution: Mitigated in [42b580b](#) :

In **KnoxNet**, the original owner-side exposure has been removed. The router no longer receives an infinite allowance over the owner balance during deployment, and `_transferOwnership` no longer grants the router an infinite allowance over the new owner balance. This eliminates the previously reported path under which the router could transfer the owner's tokens without a separate approval transaction.

The remaining approval model has not been fully minimized. The contract still grants the router a standing infinite allowance over `address(this)` through `_allowances[address(this)][routerAddress] = type(uint256).max` for swap and liquidity operations. The deployer-balance approval issue has been resolved, but the broader recommendation of avoiding unconditional infinite router approvals remains only partially addressed.

[F-2026-16002](#) - manualClogClearPercent Ignores the Configured Liquidity Split and Routes All Swap Proceeds to the Marketing Receiver - Low

Description:

In **KnoxNet**, the `manualClogClearPercent` function swaps contract-held tokens for ETH and forwards the entire ETH output to `marketingTaxReceiver`. Unlike `_autoSwapBack`, this path does not preserve any token portion for liquidity and does not allocate any ETH to `liquidityTaxReceiver`. As a result, manually cleared fee balances are distributed under a different accounting model than the one used by the normal auto-swap flow.

In **KnoxNet**, `_autoSwapBack` explicitly splits accumulated fees between marketing and liquidity. It reserves a token portion for LP creation, computes a liquidity share of received ETH, and adds liquidity through `addLiquidityETH`:

```
uint256 amountToLiquify = ((amountToSwap * totalLiquidityTaxBps) / 2) /
    combinedTaxBps;
// ...
uint256 amountETHLiquidity = ((amountETH * totalLiquidityTaxBps) / 2) /
    totalETHTax;
uint256 amountETHMarketing = amountETH - amountETHLiquidity;
// ...
router.addLiquidityETH{value: amountETHLiquidity}(
    address(this),
    amountToLiquify,
    0,
    0,
    liquidityTaxReceiver,
    block.timestamp
);
```

By contrast, `manualClogClearPercent` liquidates the selected token amount entirely into ETH and forwards all received ETH to `marketingTaxReceiver`:

```
uint256 amountToSwap = (contractTokenBalance * percent) / 100;
// ...
router.swapExactTokensForETHSupportingFeeOnTransferTokens(
    amountToSwap,
    0,
    path,
    address(this),
    block.timestamp
);
```



```
uint256 ethReceived = address(this).balance - ethBefore;
if (ethReceived > 0) {
    (bool sentMarketing, ) = marketingTaxReceiver.call{value: ethReceived}("")
;
    require(sentMarketing, "Failed to send ETH");
}
```

No token reservation, ETH split, or LP creation occurs in this manual path. Accordingly, any contract-held tokens that were originally accumulated under liquidity tax settings are treated as if they belonged entirely to marketing.

When `manualClogClearPercent` is used, liquidity-designated fee balances may be redirected fully to the marketing receiver instead of contributing to liquidity. This causes the manual recovery path to behave inconsistently with the standard auto-swap distribution model. The issue affects protocol-controlled fee allocation rather than user balances directly.

Assets:

- contracts/KnoxNet.sol [<https://github.com/Knoxnetofficial/knoxnet-contracts>]

Status:

Fixed

Classification

Impact Rate: 3/5

Likelihood Rate: 2/5

Exploitability: Dependent

Complexity: Simple

Severity: Low

Recommendations

Remediation:

- `manualClogClearPercent` should either follow the same marketing/liquidity allocation model as `_autoSwapBack` or be explicitly restricted to clearing only marketing-designated balances. If it is intended as a manual version of the standard fee-conversion flow, it should reserve the liquidity token portion, split received ETH, and call `addLiquidityETH` accordingly.
- If the current behavior is intentional, it should be documented explicitly as a separate manual marketing-only recovery path rather than presented as a general clog-clearing mechanism.

Resolution:

Fixed in [c20e980](#) :

In **KnoxNet**, `manualClogClearPercent` now follows the same marketing/liquidity allocation model as `_autoSwapBack`. The function derives `liquidityPortion` and `marketingPortion` from `accumulatedLiquidityTokens` and `accumulatedMarketingTokens`, swaps only the appropriate portion, splits the received ETH accordingly, and attempts to add liquidity through `_tryAddLiquidityETH`.

```
uint256 totalAccumulatedTokens = accumulatedMarketingTokens + accumulatedLiquidityTokens;
if (totalAccumulatedTokens == 0) {
    accumulatedMarketingTokens = contractTokenBalance;
    totalAccumulatedTokens = contractTokenBalance;
}
uint256 liquidityPortion = (accumulatedLiquidityTokens * amountToSwap) / totalAccumulatedTokens;
uint256 marketingPortion = amountToSwap - liquidityPortion;
uint256 liquidityTokensToLiquify = liquidityPortion / 2;
uint256 liquidityTokensToSwap = liquidityPortion - liquidityTokensToLiquify;
uint256 swapTokens = marketingPortion + liquidityTokensToSwap;
uint256 ethReceived = _swapTokensForETH(swapTokens);
accumulatedMarketingTokens -= marketingPortion;
accumulatedLiquidityTokens -= liquidityTokensToSwap;
uint256 amountETHLiquidity = 0;
if (liquidityTokensToSwap > 0) {
    amountETHLiquidity = (ethReceived * liquidityTokensToSwap) / swapTokens;
}
if (amountETHLiquidity > 0 && liquidityTokensToLiquify > 0) {
    (, bool added) = _tryAddLiquidityETH(liquidityTokensToLiquify, amountETHLiquidity);
    if (added) accumulatedLiquidityTokens -= liquidityTokensToLiquify;
}
```

F-2026-16003 - Zero Slippage Protection in addLiquidityETH Allows Liquidity to Be Added at Adversarially Shifted Pool Ratios - Low

Description:

In **KnoxNet**, liquidity is added through `addLiquidityETH` with both `amountTokenMin` and `amountETHMin` set to `0`. This means the operation accepts any pool ratio at execution time, even if the price has moved materially from the ratio implicitly assumed when `amountToLiquify` and `amountETHLiquidity` were calculated. As a result, liquidity can be added at an adversarially shifted price with no minimum protection.

In **KnoxNet**, `_autoSwapBack` first computes token and ETH amounts for liquidity, then calls `addLiquidityETH` without any slippage bounds:

```
if (amountToLiquify > 0) {
    router.addLiquidityETH({value: amountETHLiquidity}(
        address(this),
        amountToLiquify,
        0,
        0,
        liquidityTaxReceiver,
        block.timestamp
    ));
}
```

The zero values for `amountTokenMin` and `amountETHMin` tell the router to accept any proportion of token and ETH that matches the pool state at execution time. Because the same transaction has already executed a token sale immediately beforehand, and because external actors may further manipulate the pool ratio before a manually triggered liquidity operation is mined, the actual ratio used for LP addition can differ materially from the ratio anticipated by the contract.

When this happens, the router uses only the optimal subset of the provided assets at the manipulated price and returns the remainder to the contract. The resulting LP position is still created successfully, but at a worse ratio than intended and with no minimum-output protection.

Liquidity may be added at a materially unfavorable price if the pool ratio is manipulated or moves sharply before execution. This can reduce the efficiency of liquidity provisioning and leave residual token or ETH amounts unused in the contract. The issue affects protocol-controlled assets rather than direct user balances.

Assets:

- `contracts/KnoxNet.sol` [<https://github.com/Knoxnetofficial/knoxnet-contracts>]

Status: Fixed

Classification

Impact Rate: 3/5

Likelihood Rate: 2/5

Exploitability: Independent

Complexity: Simple

Severity: Low

Recommendations

Remediation: Non-zero minimums should be supplied to `addLiquidityETH` based on an acceptable slippage tolerance. The token and ETH amounts should be bounded relative to the expected ratio at the time of execution.

```
uint256 minToken = (amountToLiquify * 95) / 100;
uint256 minETH = (amountETHLiquidity * 95) / 100;
router.addLiquidityETH{value: amountETHLiquidity}(
    address(this),
    amountToLiquify,
    minToken,
    minETH,
    liquidityTaxReceiver,
    block.timestamp
);
```

If precise ratio enforcement is important, the contract should derive expected amounts from current pool reserves immediately before adding liquidity and reject execution when the deviation exceeds a configured tolerance.

Resolution: Fixed in `c20e980` :

In **KnoxNet**, liquidity additions no longer use zero minimums. The `_autoSwapBack` function now derives `minToken` and `minETH` from `LIQUIDITY_SLIPPAGE_BPS` and passes those bounds to `addLiquidityETH`, which prevents liquidity from being added at an unboundedly adverse ratio. The same protection is applied in `_tryAddLiquidityETH`, so the previously unrestricted slippage exposure has been addressed.

```
uint256 constant BPS_DENOMINATOR = 10000;
uint256 constant LIQUIDITY_SLIPPAGE_BPS = 500; // 5%
```

```

if (liquidityTokensToLiquify > 0 && amountETHLiquidity > 0) {
    uint256 minToken = (liquidityTokensToLiquify * (BPS_DENOMINATOR - LIQUIDITY_SLIPPAGE_BPS)) /
        BPS_DENOMINATOR;
    uint256 minETH = (amountETHLiquidity * (BPS_DENOMINATOR - LIQUIDITY_SLIPPAGE_BPS)) /
        BPS_DENOMINATOR;
    try
        router.addLiquidityETH{value: amountETHLiquidity}(
            address(this),
            liquidityTokensToLiquify,
            minToken,
            minETH,
            liquidityTaxReceiver,
            block.timestamp + ROUTER_DEADLINE_SECONDS
        )
    returns (uint256, uint256 ethUsed, uint256) {
        accumulatedLiquidityTokens -= liquidityTokensToLiquify;
        addedLiquidityTokens = liquidityTokensToLiquify;
        addedLiquidityETH = ethUsed;
    } catch {
        // skip liquidity addition on failure; tokens + ETH remain in contract
    }
}

```

[F-2026-16005](#) - Missing Lower Bounds on Transaction and Wallet Limits Allow Transfers to Be Effectively Frozen - Low

Description:

In **KnoxNet**, the owner can reduce the maximum buy limit, maximum sell limit, and maximum wallet size to arbitrarily small non-zero values. No minimum floor is enforced for these parameters. As a result, regular transfers, buys, and sells can be made impractical or effectively blocked for non-exempt users.

In **KnoxNet**, the owner can update transaction limits through `setTxLimit` and the wallet limit through `setMaxWallet`:

```
function setTxLimit(
    uint256 buyNumerator,
    uint256 sellNumerator,
    uint256 divisor
) external onlyOwner {
    require(
        buyNumerator > 0 && sellNumerator > 0 && divisor > 0 && divisor <= 100
    );
    _maxBuyTxAmount = (_totalSupply * buyNumerator) / divisor;
    _maxSellTxAmount = (_totalSupply * sellNumerator) / divisor;
}

function setMaxWallet(uint256 numerator, uint256 divisor) external onlyOwner {
    require(numerator > 0 && divisor > 0 && divisor <= 10000);
    _maxWalletSize = (_totalSupply * numerator) / divisor;
}
```

The only validation ensures that the resulting values are non-zero. This means the owner may set limits to extremely small fractions of total supply, such as $1 / 10000 = 0.01\%$ (0 in Solidity). These values are then enforced for non-exempt users during `_transferFrom` through `_enforceTxLimit` and `_enforceWalletLimit`.

Because the owner and other privileged addresses may remain exempt from these checks, the restriction can selectively burden ordinary holders while leaving administrative addresses unrestricted.

Buying, selling, or transferring tokens may become impractical for non-exempt users if the configured limits are reduced to very small values. This can effectively freeze normal token movement even though transfers are not formally paused. The issue affects core transferability of the token under owner-controlled configuration.

Assets:

- contracts/KnoxNet.sol [<https://github.com/Knoxnetofficial/knoxnet-contracts>]

Status:

Fixed

Classification**Impact Rate:**

4/5

Likelihood Rate:

2/5

Exploitability:

Dependent

Complexity:

Simple

Severity:

Low

Recommendations**Remediation:**

Reasonable lower bounds should be enforced for all transaction and wallet limits so that token transfers cannot be reduced below an acceptable operational floor.

Resolution:**Fixed in** `c20e980` :

In **KnoxNet**, minimum floors are now enforced for both transaction limits and wallet limits through the `MIN_LIMIT_BPS` constant. The `setTxLimit` and `setMaxWallet` functions compute the resulting values and require them to remain at or above $(_totalSupply * MIN_LIMIT_BPS) / BPS_DENOMINATOR$, preventing the owner from reducing these limits to arbitrarily small non-zero amounts. The previously unrestricted ability to make token movement practically unusable for non-exempt holders has therefore been constrained.

F-2026-16009 - isTxLimitExempt Simultaneously Disables Both Transaction and Wallet Limits - Low

Description:

The contract uses a single exemption flag, `isTxLimitExempt`, to bypass two different protection mechanisms: the transaction-size limit and the maximum wallet-size limit.

In `_enforceTxLimit`, the flag disables transaction limit checks for either the sender or recipient. Separately, in `_transferFrom`, the same flag is reused to decide whether the recipient should also bypass the wallet limit. As a result, granting an address exemption from transaction limits automatically exempts it from wallet-size restrictions as well.

```
if (!liquidityPools[recipient] && recipient != DEAD) {
    if (!isTxLimitExempt[recipient]) {
        _enforceWalletLimit(recipient, amount);
    }
}
```

```
function _enforceTxLimit(
    address sender,
    address recipient,
    uint256 amount
) internal view {
    if (isTxLimitExempt[sender] || isTxLimitExempt[recipient]) return;
    require(
        amount <= (liquidityPools[sender] ? _maxBuyTxAmount : _maxSellTxAmount),
        "Amount exceeds the tx limit."
    );
    require(blacklist[sender] == 0, "Wallet blacklisted!");
}
```

This coupling reduces administrative granularity and can lead to broader privilege grants than intended. For example, if the owner wants to allow a specific address to exceed per-transaction limits for operational reasons, that address will also be able to accumulate tokens beyond the configured max wallet limit.

While this does not directly expose user funds, it weakens the intended separation of controls and may undermine token distribution constraints.

Assets:

- `contracts/KnoxNet.sol` [<https://github.com/Knoxnetofficial/knoxnet-contracts>]

Status: Fixed

Classification

Impact Rate: 2/5

Likelihood Rate: 3/5

Exploitability: Independent

Complexity: Simple

Severity: Low

Recommendations

Remediation: Introduce a separate exemption mapping for the wallet limit, such as `isWalletLimitExempt`, and use it exclusively for `_enforceWalletLimit`. This keeps transaction-limit and wallet-limit permissions independent and makes administrative intent explicit.

Resolution: Fixed in `c20e980` :

In **KnoxNet**, wallet-limit exemptions have been separated from transaction-limit exemptions through the introduction of the dedicated `isWalletLimitExempt` mapping. The `_transferFrom` function now checks `isWalletLimitExempt[recipient]` before applying `_enforceWalletLimit`, while `_enforceTxLimit` continues to rely on `isTxLimitExempt`. The prior coupling between transaction-limit and wallet-limit bypass permissions has therefore been removed.

F-2026-15976 - Missing Zero-Address Validation in setTaxReceivers Can Permanently Misroute Tax Proceeds and Liquidity Assets - Info

Description:

In **KnoxNet**, the `setTaxReceivers` function accepts arbitrary addresses for both tax receiver roles without validating that they are non-zero. If either receiver is set to `address(0)`, subsequent tax distribution and liquidity operations may direct ETH or liquidity positions to an unusable destination. This creates a configuration path that can permanently misroute protocol-controlled assets.

In **KnoxNet**, the `setTaxReceivers` function stores both receiver addresses directly:

```
function setTaxReceivers(
    address _liquidityTaxReceiver,
    address _marketingTaxReceiver
) external onlyOwner {
    liquidityTaxReceiver = payable(_liquidityTaxReceiver);
    marketingTaxReceiver = payable(_marketingTaxReceiver);
}
```

These values are later used in tax distribution and liquidity operations. Marketing ETH is forwarded through a low-level call:

```
(bool sentMarketing, ) = marketingTaxReceiver.call{value: amountETHMarketing}(
    ""
);
Liquidity positions are assigned to liquidityTaxReceiver during addLiquidityETH:
router.addLiquidityETH{value: amountETHLiquidity}(
    address(this),
    amountToLiquify,
    0,
    0,
    liquidityTaxReceiver,
    block.timestamp
);
```

If `marketingTaxReceiver` is `address(0)`, ETH may be transferred to the zero address. If `liquidityTaxReceiver` is `address(0)`, the resulting liquidity tokens are minted to the zero address and become irrecoverable.

Assets:

- `contracts/KnoxNet.sol` [<https://github.com/Knoxnetofficial/knoxnet-contracts>]

Status: Fixed

Classification

Impact Rate: 3/5

Likelihood Rate: 1/5

Exploitability: Dependent

Complexity: Simple

Severity: Info

Recommendations

Remediation: Both receiver addresses should be validated against `address(0)` before being stored.

Resolution: Fixed in `**c20e980**`:

In **KnoxNet**, `setTaxReceivers` now validates both receiver addresses before updating storage. The function reverts if either `newLiquidityTaxReceiver` or `newMarketingTaxReceiver` is the zero address, which prevents marketing ETH or liquidity positions from being routed to an unusable destination. The misconfiguration path described in the finding has therefore been removed.

F-2026-15979 - Use of tx.origin in Blacklist Event Produces Unreliable Caller Attribution - Info

Description:

In **KnoxNet**, the blacklist event emitted by `updateBlacklist` records `tx.origin` instead of the direct caller. This does not affect contract state, but it causes the event log to attribute blacklist actions to the externally owned account that originated the transaction rather than the contract or multisig that actually invoked the function. As a result, event-based audit trails may report misleading operator information.

In **KnoxNet**, `updateBlacklist` emits the `WalletRestricted` event as follows:

```
function updateBlacklist(
    address[] calldata _wallets,
    bool _blacklist
) external onlyTrustedOperator {
    for (uint i = 0; i < _wallets.length; i++) {
        if (_blacklist) {
            blacklistCount++;
            emit WalletRestricted(tx.origin, _wallets[i], block.number);
        }
        // ...
    }
}
```

When the function is called through a multisig, relay, or other intermediary contract, `tx.origin` resolves to the original externally owned account that initiated the outer transaction, not the immediate caller authorized by `onlyTrustedOperator`. The emitted event therefore does not reliably represent the actor that executed the blacklist action in contract terms.

Off-chain monitoring, analytics, and audit trails may attribute blacklist actions to the wrong address. This can make administrative activity harder to trace accurately, especially when privileged operations are executed through multisigs or operational contracts.

Assets:

- contracts/KnoxNet.sol [<https://github.com/Knoxnetofficial/knoxnet-contracts>]

Status:

Fixed

Classification

Impact Rate:	1/5
Likelihood Rate:	1/5
Exploitability:	Independent
Complexity:	Simple
Severity:	Info

Recommendations

Remediation: The event should record `msg.sender` instead of `tx.origin` so that logs reflect the direct authorized caller.

Resolution: **Fixed in `c20e980` :**

In **KnoxNet**, `updateBlacklist` now emits `WalletRestricted` with `_msgSender()` instead of `tx.origin`. The event log therefore records the direct authorized caller, including multisigs or intermediary operational contracts, rather than the originating externally owned account. The misleading attribution described in the finding has been removed.

```
function updateBlacklist(
    address[] calldata _wallets,
    bool _blacklist
) external onlyTrustedOperator {
    for (uint i = 0; i < _wallets.length; i++) {
        bool isCurrentlyBlacklisted = blacklist[_wallets[i]] != 0;
        if (_blacklist) {
            if (!isCurrentlyBlacklisted) {
                blacklistCount++;
                emit WalletRestricted(_msgSender(), _wallets[i], block.number)
            }
            blacklist[_wallets[i]] = block.number;
        }
    }
}
```

[F-2026-15982](#) - Missing Zero-Address Validation in airdrop Allows Silent Token Burning and Distorts Circulating Supply Reporting - Info

Description:

In **KnoxNet**, the `airdrop` function forwards transfers through `_basicTransfer` without validating that each recipient is non-zero. As a result, tokens can be airdropped to `address(0)` instead of being rejected. This creates an administrative path for silent token burning and causes the zero address balance to be included in the contract's circulating supply calculation.

In **KnoxNet**, `airdrop` iterates over user-supplied recipients and directly calls `_basicTransfer`:

```
function airdrop(
    address[] calldata addresses,
    uint256[] calldata amounts
) external onlyOwner {
    // ...
    for (uint i = 0; i < addresses.length; i++) {
        if (!liquidityPools[addresses[i]] && !isLiquidityCreator[addresses[i]])
    } {
        _basicTransfer(from, addresses[i], amounts[i] * (10 ** _decimals))
    ;
    }
    }
}
```

Unlike `_transferFrom`, this path does not validate `recipient != address(0)`. The internal `_basicTransfer` function likewise performs only balance updates:

```
function _basicTransfer(
    address sender,
    address recipient,
    uint256 amount
) internal returns (bool) {
    _balances[sender] = _balances[sender] - amount;
    _balances[recipient] = _balances[recipient] + amount;
    emit Transfer(sender, recipient, amount);
    return true;
}
```

If `address(0)` is passed in the airdrop list, the transfer succeeds and the zero-address balance increases. Since **KnoxNet** computes circulating supply as `_totalSupply - (balanceOf(DEAD) + balanceOf(ZERO))`, this

administrative burn path directly affects reported circulating supply without any explicit burn function or validation.

Tokens may be irreversibly transferred to `address(0)` through `airdrop` without reverting. This can unintentionally burn tokens and reduce the reported circulating supply. The behavior is silent and may be triggered by operator error in the recipient list.

Assets:

- `contracts/KnoxNet.sol` [<https://github.com/Knoxnetofficial/knoxnet-contracts>]

Status:

Fixed

Classification

Impact Rate: 2/5

Likelihood Rate: 2/5

Exploitability: Dependent

Complexity: Simple

Severity: Info

Recommendations

Remediation: Each airdrop recipient should be validated against `address(0)` before `_basicTransfer` is called. Invalid recipients should cause the transaction to revert rather than silently burn tokens.

Resolution: Fixed in `c20e980` :

In **KnoxNet**, the `airdrop` function now validates each recipient before executing `_basicTransfer`. The function reverts when any entry in `addresses` is the zero address, which removes the prior administrative path that allowed silent token burning through `address(0)` and prevents unintended changes to the reported circulating supply.

[F-2026-15983](#) - Inconsistent Amount Units Across Administrative and View Functions Increase the Risk of Off-Chain Misinterpretation and Misconfiguration - Info

Description:

In **KnoxNet**, multiple externally accessible functions use different amount-unit conventions without enforcing a consistent interface pattern. Some functions accept human-readable whole-token values and internally scale them by `10 ** _decimals`, while core ERC20 behavior and internal accounting operate in base units. Other view functions return scaled-down whole-token values instead of raw base-unit amounts. This inconsistency increases the risk of operator mistakes, incorrect UI assumptions, and off-chain integration errors.

The issue appears in several places in **KnoxNet**.

The `airdrop` function expects whole-token inputs and scales them internally:

```
function airdrop(
    address[] calldata addresses,
    uint256[] calldata amounts
) external onlyOwner {
    // ...
    _basicTransfer(from, addresses[i], amounts[i] * (10 ** _decimals));
}
```

The `configureAutoSwap` function mixes conventions in the same call. `_denominator` is ratio-based, while `_minTokenBalance` is interpreted as a whole-token value and then scaled internally:

```
function configureAutoSwap(
    bool _enabled,
    uint256 _denominator,
    uint256 _minTokenBalance
) external onlyOwner {
    // ...
    autoSwapTokenThreshold = _totalSupply / _denominator;
    autoSwapMinTokenBalance = _minTokenBalance * (10 ** _decimals);
}
```

The limit getter functions move in the opposite direction and return whole-token values rather than the raw base-unit values used internally:

```
function maxBuyTxTokens() external view returns (uint256) {
    return _maxBuyTxAmount / (10 ** _decimals);
}
```



```

}
function maxSellTxTokens() external view returns (uint256) {
    return _maxSellTxAmount / (10 ** _decimals);
}
function maxWalletTokens() external view returns (uint256) {
    return _maxWalletSize / (10 ** _decimals);
}

```

Taken together, these interfaces do not follow a single unit convention. A caller may reasonably assume that all amounts are base-unit ERC20 values, or that all helper functions consistently use whole-token semantics, but neither assumption holds across the contract.

Assets:

- contracts/KnoxNet.sol [<https://github.com/Knoxnetofficial/knoxnet-contracts>]

Status:

Accepted

Classification

Impact Rate: 4/5

Likelihood Rate: 1/5

Exploitability: Dependent

Complexity: Simple

Severity: Info

Recommendations

Remediation: A single unit convention should be adopted across externally visible functions. Preferably, all externally supplied and returned amounts should use raw ERC20 base units, with any human-readable conversion handled off-chain.

Resolution: **Accepted:**

In **KnoxNet**, externally accessible functions still use mixed amount-unit conventions. The `airdrop` function accepts whole-token values and scales them internally, `configureAutoSwap` mixes a ratio-based `_denominator` with a whole-token `_minTokenBalance` that is later scaled to base units, and the `maxBuyTxTokens`, `maxSellTxTokens`, and `maxWalletTokens` view functions return scaled-down whole-token values rather than raw base-unit amounts. No unified external amount convention has been introduced, and the inconsistency described in the finding remains present.

F-2026-15986 - Mid-Section State Variable Declaration Reduces Structural Consistency - Info

Description:

In **KnoxNet**, the `autoSwapSellCapPercent` state variable is declared in the middle of the execution logic, immediately before `_autoSwapBack`, rather than alongside the rest of the contract's state configuration. This does not affect runtime behavior, but it breaks structural consistency and makes the contract harder to review. As a result, important configurable parameters may be overlooked during maintenance or security analysis.

Most state variables in **KnoxNet** are declared together near the top of the contract. However, `autoSwapSellCapPercent` is introduced later, directly above the `_autoSwapBack` function:

```
// Max share of the user's sell amount the contract may swap in one go (10%).
uint256 public autoSwapSellCapPercent = 10;
function _autoSwapBack(uint256 amount) internal swapping {
    // ...
}
```

Since this variable controls swap behavior and is updated through `setAutoSwapSellCapPercent`, it belongs to the same configuration group as the other auto-swap parameters, such as `autoSwapEnabled`, `autoSwapTokenThreshold`, `autoSwapMinTokenBalance`, and `autoSwapPerBlockLimit`. Its current placement fragments the configuration surface and reduces codebase consistency.

The issue does not introduce incorrect behavior or exploitable logic. However, contract review, parameter auditing, and future maintenance may become less reliable because a relevant configuration variable is declared outside the main state section.

Assets:

- contracts/KnoxNet.sol [<https://github.com/Knoxnetofficial/knoxnet-contracts>]

Status:

Fixed

Classification

Impact Rate: 1/5

Likelihood Rate: 1/5

Exploitability: Independent

Complexity: Simple

Severity: Info

Recommendations

Remediation: The `autoSwapSellCapPercent` variable should be moved into the primary state-variable section near the other auto-swap configuration parameters.

Resolution: **Fixed in** `c20e980` :

In **KnoxNet**, `autoSwapSellCapPercent` has been moved into the main state-variable section alongside the other auto-swap configuration parameters. The variable now appears next to `autoSwapEnabled`, `autoSwapTokenThreshold`, and `autoSwapMinTokenBalance`, which restores structural consistency and makes the auto-swap configuration surface easier to review and maintain.

[F-2026-15987](#) - Missing Events for Administrative Configuration Changes Reduces On-Chain Observability - Info

Description:

In **KnoxNet**, multiple privileged administrative functions update critical configuration without emitting dedicated events. As a result, important changes to operator permissions, trading status, tax behavior, liquidity-pool classification, and auto-swap settings cannot be tracked efficiently through the event log. This reduces on-chain observability and complicates off-chain monitoring and forensic analysis.

Several state-changing functions in **KnoxNet** mutate operationally important configuration without emitting a corresponding event, including:

```
function setTrustedOperator(address _operator, bool _enabled) external onlyOwner {
    trustedOperators[_operator] = _enabled;
}

function openTrading() external onlyTrustedOperator {
    isTradingEnabled = true;
    autoSwapEnabled = true;
}

function addLiquidityPool(address lp, bool isPool) external onlyOwner {
    liquidityPools[lp] = isPool;
}
```

The same issue applies to other configuration setters such as

`setAutoSwapPerBlockLimit`, `setAutoSwapSellCapPercent`, `toggleTransferTax`, `setTaxReceivers`, `configureAutoSwap`, `setIsTaxExempt`, and `setIsTxLimitExempt`.

Since these functions affect transfer restrictions, taxation, auto-swap behavior, and privileged access, the absence of dedicated events weakens operational transparency.

Off-chain systems cannot reliably monitor critical configuration changes through event subscriptions alone. Administrative actions may therefore be harder to detect, index, or audit in real time. This primarily affects transparency and operational monitoring rather than core fund safety.

Assets:

- contracts/KnoxNet.sol [<https://github.com/Knoxnetofficial/knoxnet-contracts>]

Status:

Fixed

Classification

Impact Rate:	1/5
Likelihood Rate:	3/5
Exploitability:	Independent
Complexity:	Simple
Severity:	Info

Recommendations

Remediation: Dedicated events should be emitted for each critical administrative state change.

Resolution: **Fixed in** `c20e980` :

In **KnoxNet**, dedicated events have been added for the privileged administrative actions identified in the finding. Critical configuration updates such as trusted-operator changes, trading activation, liquidity-pool classification, tax configuration, tax-receiver updates, auto-swap configuration, exemption changes, and transfer-tax toggling now emit corresponding events. The event log can therefore be used to monitor these state changes directly.

F-2026-15988 - Co-Locating All Contracts and Interfaces in a Single Source File Reduces Reviewability and Maintainability - Info

Description:

In **KnoxNet**, the main token contract, interfaces, and inlined utility and ownership components are all defined in a single Solidity source file. This does not directly affect runtime behavior, but it reduces codebase modularity and makes security review, maintenance, and change tracking more error-prone. As a result, standard components and custom logic become harder to distinguish and audit independently.

The `KnoxNet.sol` source file contains multiple distinct components in one place, including the `Address` library, `Context`, `IERC20`, `IDEXFactory`, `IDEXRouter`, `Ownable`, and the main **KnoxNet** contract. In particular, standard-style infrastructure and custom business logic are co-located in a single compilation unit:

```
library Address { /* ... */ }
abstract contract Context { /* ... */ }
interface IERC20 { /* ... */ }
interface IDEXFactory { /* ... */ }
interface IDEXRouter { /* ... */ }
abstract contract Ownable is Context { /* ... */ }
contract KnoxNet is IERC20, Ownable {
    // custom token logic
}
```

This structure makes it harder to separate third-party or standard code from project-specific logic during review. It also increases the likelihood that future changes to one component affect unrelated parts of the file without being noticed promptly in code review or remediation cycles.

The issue does not create incorrect on-chain behavior by itself. However, auditability, maintainability, and change traceability are reduced, particularly when standard components are inlined alongside custom token logic. This may increase the risk of review blind spots during future modifications.

Assets:

- `contracts/KnoxNet.sol` [<https://github.com/Knoxnetofficial/knoxnet-contracts>]

Status:

Accepted

Classification

Impact Rate:	1/5
Likelihood Rate:	2/5
Exploitability:	Independent
Complexity:	Simple
Severity:	Info

Recommendations

Remediation: Standard libraries, interfaces, and reusable base contracts should be split into separate source files and imported into the main token contract. Third-party components should preferably be referenced from well-known upstream packages instead of being inlined unless there is a documented reason to vendor them locally.

```
import "@openzeppelin/contracts/access/Ownable.sol";
import "@openzeppelin/contracts/utils/Address.sol";
import "../interfaces/IERC20.sol";
import "../interfaces/IDEXFactory.sol";
import "../interfaces/IDEXRouter.sol";

contract KnoxNet is IERC20, Ownable {
    // ...
}
```

Custom logic should remain isolated in the main contract file, while infrastructure and interface definitions should be grouped by responsibility in separate modules.

Resolution:

Accepted:

In **KnoxNet**, the main token logic, interfaces, and inlined utility and ownership components remain defined within a single Solidity source file. No modularization into separate files or imported standard dependencies has been introduced. The structural issue therefore remains unresolved and has been accepted.

[F-2026-15989](#) - Unused using Address for address Declaration Introduces Dead Code and Reduces Code Clarity - Info

Description:

In **KnoxNet**, the `Address` library is attached to the `address` type through `using Address for address`, but no library methods are invoked in the custom contract logic. This does not affect runtime behavior directly, but it introduces dead code and suggests a dependency that is not actually used. As a result, the contract appears more complex than it is and becomes slightly harder to review and maintain.

In **KnoxNet**, the following declaration is present near the start of the main contract:

```
contract KnoxNet is IERC20, Ownable {
    using Address for address;
    // ...
}
```

However, no `address` value in **KnoxNet** calls `Address` library methods such as `isContract`, `sendValue`, `functionCall`, `functionStaticCall`, or `functionDelegateCall`. ETH transfers and external calls are instead implemented directly through low-level `.call` and router interactions.

Because the declaration is unused, the presence of the inlined `Address` library suggests a functional dependency that does not exist in practice. This reduces readability and may cause reviewers or maintainers to spend time tracing code paths that are never exercised.

No incorrect on-chain behavior is introduced by this issue. However, dead code increases audit surface area, reduces clarity, and may slightly increase deployment size when inlined library code is retained without need.

Assets:

- `contracts/KnoxNet.sol` [<https://github.com/Knoxnetofficial/knoxnet-contracts>]

Status:

Fixed

Classification

Impact Rate: 1/5

Likelihood Rate: 1/5

Exploitability: Independent

Complexity: Simple

Severity: Info

Recommendations

Remediation: The unused `using Address for address` declaration should be removed. If the `Address` library is otherwise unnecessary, the inlined library itself should also be removed or replaced with a standard import only when needed.

Resolution: **Fixed in c20e980 :**

In **KnoxNet**, the unused `using Address for address` declaration has been removed from the main contract. No `Address` library methods are invoked by the custom token logic, so the unused attachment no longer suggests a dependency that does not exist in practice. The dead declaration identified in the finding is therefore resolved.

F-2026-15990 - Non-Mutable Configuration Values Are Stored in Regular Storage Instead of constant or immutable - Info

Description:

In **KnoxNet**, `_totalSupply` and `routerAddress` are assigned once and never modified, yet both are stored as regular storage variables. This does not affect functional correctness, but it introduces unnecessary storage reads and increases gas consumption for operations that depend on these values. As a result, the contract incurs avoidable runtime overhead for values that could be embedded directly in bytecode.

In **KnoxNet**, both values are initialized inline and remain unchanged throughout the contract lifecycle:

```
uint256 _totalSupply = 1_000_000_000 * (10 ** _decimals);  
address routerAddress = 0x7a250d5630B4cF539739dF2C5dAcb4c659F2488D;
```

Neither variable has a setter, and neither is reassigned after deployment. Despite this, both occupy storage slots and are read through SLOAD operations when referenced by runtime logic such as constructor setup, limit configuration, auto-swap configuration, and supply-related calculations.

Because these values are static, `constant` or `immutable` would be more appropriate. `_totalSupply` can be declared `constant`, while `routerAddress` can be declared either `constant` or `immutable` depending on whether configurability at deployment is desired.

No incorrect token behavior is introduced by this issue. However, the contract pays unnecessary gas for reading values that never change. This increases execution cost and reduces code-quality efficiency for a fixed-configuration token implementation.

Assets:

- contracts/KnoxNet.sol [<https://github.com/Knoxnetofficial/knoxnet-contracts>]

Status:

Fixed

Classification

Impact Rate: 1/5

Likelihood Rate: 1/5

Exploitability: Independent

Complexity: Simple

Severity: Info

Recommendations

Remediation:

- Static values should be declared as `constant` where possible, or `immutable` if they must be assigned once during deployment.
- If router migration must be supported in the future, the migration path should be implemented as a dedicated function that revokes the old allowance, validates the new router, sets a new allowance, and is gated behind a timelock. This approach avoids inheriting stale approvals while preserving migration capability.

Resolution:

Fixed in `c20e980` :

In **KnoxNet**, both `_totalSupply` and `routerAddress` have been converted from mutable storage variables to constants. Since neither value is reassigned after deployment, the prior storage-backed configuration has been removed and the values are now embedded directly in bytecode. The unnecessary storage overhead identified in the finding is therefore resolved

```
uint256 constant _totalSupply = 1_000_000_000 * (10 ** _decimals);  
address constant routerAddress = 0x7a250d5630B4cF539739dF2C5dAcb4c659F2488D;
```

F-2026-15991 - Unnamed WalletRestricted Event Parameters Reduce Log Readability and Off-Chain Decoding Clarity - Info

Description:

In **KnoxNet**, the `WalletRestricted` event is declared without parameter names. This does not affect event emission or on-chain behavior, but it reduces log readability and makes off-chain decoding less clear in explorers, indexers, and analytics tools. As a result, blacklist-related event data becomes harder to interpret consistently.

In **KnoxNet**, the event is declared as follows:

```
event WalletRestricted(address, address, uint256);
```

Because the parameters are unnamed, off-chain tools typically expose them as generic fields such as `param0`, `param1`, and `param2` rather than meaningful identifiers. This contrasts with the other events in **KnoxNet**, which are declared with descriptive names:

```
event TaxDistributed(  
    uint256 marketingETH,  
    uint256 liquidityETH,  
    uint256 liquidityTokens  
);
```

The missing names do not alter the emitted data, but they do make it less obvious which address represents the caller and which represents the restricted wallet.

No incorrect contract behavior is introduced by this issue. However, event logs become less readable and more error-prone to interpret for off-chain consumers. This weakens observability and slightly increases operational friction during monitoring or forensic review.

Assets:

- `contracts/KnoxNet.sol` [<https://github.com/Knoxnetofficial/knoxnet-contracts>]

Status:

Fixed

Classification

Impact Rate: 1/5

Likelihood Rate: 1/5

Exploitability: Independent

Complexity: Simple

Severity: Info

Recommendations

Remediation: The `WalletRestricted` event should be updated to use descriptive parameter names. Indexed parameters may also be added if blacklist monitoring is expected to rely on log filtering.

```
event WalletRestricted(  
    address indexed caller,  
    address indexed wallet,  
    uint256 blockNumber  
);
```

Event declarations should follow the same naming convention already used by the other events in **KnoxNet** to keep the logging interface consistent and easier to consume.

Resolution: Fixed in `c20e980` :

In **KnoxNet**, the `WalletRestricted` event now uses descriptive parameter names and indexed address fields. The event declaration clearly identifies the caller, the affected wallet, and the block number, which improves log readability and makes blacklist-related activity easier to filter and interpret off-chain.

```
event WalletRestricted(address indexed caller, address indexed wallet, uint256  
    blockNumber);
```

F-2026-15992 - Missing Input Validation in recoverStuckETH Can Cause Misconfiguration Reverts or Irrecoverable ETH Transfers - Info

Description:

In **KnoxNet**, the `recoverStuckETH` function does not validate either `amountPercentage` or the recipient address `adr`. As a result, invalid input values may cause unnecessary transaction failure or allow ETH to be sent to `address(0)`. This weakens administrative safety and makes the recovery function more error-prone than necessary.

In **KnoxNet**, `recoverStuckETH` forwards a percentage of the contract's ETH balance to an arbitrary address:

```
function recoverStuckETH(
    uint256 amountPercentage,
    address adr
) external onlyTrustedOperator {
    uint256 amountETH = address(this).balance;
    if (amountETH > 0) {
        (bool sent, ) = adr.call{value: (amountETH * amountPercentage) / 100}("");
        require(sent, "Failed to transfer funds");
    }
}
```

No upper bound is enforced on `amountPercentage`. If a value greater than `100` is supplied, the computed transfer amount may exceed the contract's ETH balance, causing the low-level call to fail. Likewise, `adr` is not checked against `address(0)`, so ETH may be transferred to the zero address if the call succeeds.

These cases do not enable external exploitation because the function is privileged, but they create avoidable failure modes and configuration hazards in an administrative recovery path.

Administrative recovery attempts may revert when `amountPercentage` exceeds `100`, even though the intended operation is conceptually simple. ETH may also be irreversibly sent to `address(0)` if an invalid recipient address is provided. The issue primarily affects operational safety rather than core protocol logic.

Assets:

- contracts/KnoxNet.sol [<https://github.com/Knoxnetofficial/knoxnet-contracts>]

Status:

Fixed

Classification

Impact Rate:	3/5
Likelihood Rate:	1/5
Exploitability:	Dependent
Complexity:	Simple
Severity:	Info

Recommendations

Remediation: The function should validate both the percentage range and the recipient address before attempting the transfer.

Resolution: **Fixed in `c20e980` :**

In **KnoxNet**, `recoverStuckETH` now validates both the percentage input and the recipient address before attempting the transfer. The function requires `amountPercentage` to be within `1` to `100` and rejects the zero address as a recipient. The avoidable failure modes and misrouting path described in the finding have therefore been removed.

```
function recoverStuckETH(
    uint256 amountPercentage,
    address recipient
) external onlyTrustedOperator {
    require(amountPercentage > 0 && amountPercentage <= 100, "Percent must be 1-100%");
    require(recipient != address(0), "Recipient is zero address");
    uint256 amountETH = address(this).balance;
    if (amountETH > 0) {
        (bool sent, ) = recipient.call{value: (amountETH * amountPercentage) / 100}("");
        require(sent, "Failed to transfer funds");
    }
}
```

[F-2026-15993](#) - Redundant amount > 0 Condition in Auto-Swap Trigger Adds Unnecessary Logic - Info

Description:

In **KnoxNet**, the auto-swap trigger condition includes an `amount > 0` check even though zero-value transfers are already rejected earlier in the transfer flow. This does not alter execution behavior, but it introduces redundant logic and slightly reduces code clarity. As a result, the condition suggests an additional safety check that is not functionally reachable.

In **KnoxNet**, `_transferFrom` rejects zero-value transfers at the start of execution:

```
require(amount > 0, "Amount must be > zero");
```

Later in the same function, the auto-swap trigger repeats the same condition:

```
if (_shouldApplyTax(sender, recipient)) {  
    amountReceived = _applyTax(recipient, amount);  
    if (_shouldAutoSwap(recipient) && amount > 0) _autoSwapBack(amount);  
}
```

Because the earlier `require` guarantees `amount` is strictly greater than zero for any successful execution path, the later `amount > 0` branch condition is always true whenever reached. The additional check is therefore redundant and has no effect on control flow.

No incorrect on-chain behavior is introduced by this issue. However, redundant conditions increase code noise and make the control flow appear more complex than it actually is. This slightly reduces readability and maintainability.

Assets:

- `contracts/KnoxNet.sol` [<https://github.com/Knoxnetofficial/knoxnet-contracts>]

Status:

Fixed

Classification

Impact Rate:	1/5
Likelihood Rate:	1/5
Exploitability:	Independent

Complexity: Simple

Severity: Info

Recommendations

Remediation: The redundant `amount > 0` condition should be removed from the auto-swap trigger.

```
if (_shouldApplyTax(sender, recipient)) {  
    amountReceived = _applyTax(recipient, amount);  
    if (_shouldAutoSwap(recipient)) _autoSwapBack(amount);  
}
```

Checks that are already guaranteed by earlier `require` statements should not be repeated unless they protect a genuinely distinct execution path.

Resolution: Fixed in `c20e980` :

In **KnoxNet**, the redundant `amount > 0` condition has been removed from the auto-swap trigger in `_transferFrom`. Since zero-value transfers are already rejected earlier in the function, `_autoSwapBack` is now called based only on `_shouldAutoSwap(recipient)`. The unreachable condition identified in the finding is therefore no longer present.

```
if (_shouldApplyTax(sender, recipient)) {  
    amountReceived = _applyTax(sender, recipient, amount);  
    if (_shouldAutoSwap(recipient)) _autoSwapBack(amount);  
}
```

[F-2026-15994](#) - Inconsistent Use of `_msgSender()` and `msg.sender` Reduces Call-Context Consistency - Info

Description:

In **KnoxNet**, sender identity is resolved inconsistently across the contract, with some paths using `_msgSender()` and others using `msg.sender` directly. This does not change current behavior because **Context** returns `msg.sender` unchanged, but it introduces inconsistency in how call context is modeled. As a result, the contract becomes harder to maintain and less adaptable to future extensions that rely on contextual sender abstraction.

In **KnoxNet**, `onlyTrustedOperator` mixes both approaches in the same condition:

```
modifier onlyTrustedOperator() {
    require(
        trustedOperators[_msgSender()] || msg.sender == owner(),
        "Caller is not a trusted operator"
    );
    _;
}
```

Other functions use `msg.sender` directly instead of `_msgSender()`, for example:

```
function approve(
    address spender,
    uint256 amount
) public override returns (bool) {
    _allowances[msg.sender][spender] = amount;
    emit Approval(msg.sender, spender, amount);
    return true;
}
```

Because **KnoxNet** inherits **Context**, `_msgSender()` is the intended abstraction point for caller resolution. While both expressions are equivalent in the current implementation, mixing them creates ambiguity about whether contextual sender abstraction is actually intended to be supported consistently.

Assets:

- `contracts/KnoxNet.sol` [<https://github.com/Knoxnetofficial/knoxnet-contracts>]

Status:

Fixed

Classification

Impact Rate:	1/5
Likelihood Rate:	1/5
Exploitability:	Independent
Complexity:	Simple
Severity:	Info

Recommendations

Remediation: A single sender-resolution convention should be used consistently throughout the contract. Since **KnoxNet** already inherits **Context**, `_msgSender()` should be preferred wherever caller identity is needed.

Resolution: **Fixed in c20e980 :**

In **KnoxNet**, caller resolution has been standardized on `_msgSender()` throughout the contract logic. The previous mixed use of `_msgSender()` and direct `msg.sender` references in operational code is no longer present, and the only remaining `msg.sender` occurrence is inside the inherited `Context` abstraction itself.

F-2026-15995 - Public Visibility of `_shouldApplyTax` Conflicts With Internal Naming Convention and Reduces Interface Clarity - Info

Description:

In **KnoxNet**, the `_shouldApplyTax` function is declared `public` even though its underscore-prefixed naming convention indicates internal-use semantics. This does not create incorrect runtime behavior, but it makes the contract interface less clear and introduces inconsistency with the rest of the internal helper design. As a result, external consumers may misinterpret an implementation detail as an intended public API surface.

In **KnoxNet**, `_shouldApplyTax` is declared as follows:

```
function _shouldApplyTax(
    address sender,
    address recipient
) public view returns (bool) {
    if (!transferTax && !liquidityPools[recipient] && !liquidityPools[sender])
        return false;
    return !isTaxExempt[sender] && !isTaxExempt[recipient];
}
```

The underscore prefix typically denotes an internal helper, and this convention is used consistently elsewhere in **KnoxNet** for functions such as `_transferFrom`, `_basicTransfer`, `_enforceTxLimit`, `_applyTax`, `_shouldAutoSwap`, `_isLaunched`, and `_recordLaunch`, all of which are declared `internal`. The `public` visibility of `_shouldApplyTax` therefore stands out as an exception and exposes an implementation-level helper as part of the external contract interface.

Assets:

- `contracts/KnoxNet.sol` [<https://github.com/Knoxnetofficial/knoxnet-contracts>]

Status:

Fixed

Classification

Impact Rate: 1/5

Likelihood Rate: 1/5

Exploitability: Independent

Complexity: Simple

Recommendations

Remediation:

The function should either be made `internal` to match its naming convention, or renamed without a leading underscore if it is intended to be part of the public interface.

```
function _shouldApplyTax(  
    address sender,  
    address recipient  
) internal view returns (bool) {  
    if (!transferTax && !liquidityPools[recipient] && !liquidityPools[sender])  
        return false;  
    return !isTaxExempt[sender] && !isTaxExempt[recipient];  
}
```

If external visibility is intentionally required for monitoring or UI purposes, a clearer public-facing name should be introduced instead.

Resolution:

Fixed in `c20e980`:

In **KnoxNet**, `_shouldApplyTax` is now declared `internal`, which aligns its visibility with its underscore-prefixed internal-helper naming convention.

F-2026-15998 - Abbreviated Parameter Names Reduce Interface Clarity and Code Readability - Info

Description:

In **KnoxNet**, several externally callable functions use abbreviated or non-descriptive parameter names such as `adr`, `lp`, and `rate`. This does not affect runtime behavior, but it reduces interface clarity and makes the administrative API harder to interpret during review, integration, and maintenance. As a result, operator-facing functions become more error-prone to use correctly.

In **KnoxNet**, multiple public or external functions expose shortened parameter names that do not clearly communicate intent:

```
function recoverStuckETH(
    uint256 amountPercentage,
    address adr
) external onlyTrustedOperator { /* ... */ }
function addLiquidityPool(address lp, bool isPool) external onlyOwner { /* ..
. */ }
function setAutoSwapPerBlockLimit(uint256 rate) external onlyOwner { /* ... *
/ }
```

While these names are syntactically valid, they are less descriptive than the rest of the contract's generally readable naming style. For example, `adr` does not clearly indicate whether it represents a recipient, treasury, recovery target, or arbitrary external address; `lp` is less explicit than `pool` or `liquidityPool`; and `rate` does not convey that the value represents a per-block auto-swap limit rather than a percentage or tax rate.

This issue is most noticeable in privileged administrative functions, where parameter meaning directly affects operational actions.

The external interface becomes harder to understand and more prone to operator confusion, especially during manual administration or off-chain integration.

Assets:

- contracts/KnoxNet.sol [<https://github.com/Knoxnetofficial/knoxnet-contracts>]

Status:

Fixed

Classification

Impact Rate:

1/5

Likelihood Rate: 1/5

Exploitability: Independent

Complexity: Simple

Severity: Info

Recommendations

Remediation: Administrative and externally callable functions should use descriptive parameter names that clearly communicate purpose.

Resolution: **Fixed in c20e980 :**

In **KnoxNet**, the abbreviated parameter names identified in the finding have been replaced with descriptive names in the externally callable administrative functions. `adr` has been changed to `recipient`, `lp` to `liquidityPool`, and `rate` to `maxSwapsPerBlock`, which makes the administrative interface clearer and reduces ambiguity during review and operation.

[F-2026-15999](#) - Equal Token Split for Liquidity Addition Produces Suboptimal Pairing After Price Impact From the Preceding Swap - Info

Description:

In **KnoxNet**, the `_autoSwapBack` function reserves exactly half of the liquidity portion as tokens and sells the other half for ETH. This 50/50 split is calculated before the swap, but the swap itself moves the pool price. After execution, the reserved tokens and the received ETH no longer represent equal value, causing `addLiquidityETH` to leave a residual amount unused. As a result, each auto-swap cycle slightly under-utilizes the allocated liquidity resources.

In **KnoxNet**, the liquidity token reservation is computed as an exact half:

```
uint256 amountToLiquify = ((amountToSwap * totalLiquidityTaxBps) / 2) /  
    combinedTaxBps;  
amountToSwap -= amountToLiquify;
```

Then the remaining `amountToSwap` (including the other liquidity half) is sold for ETH:

```
router.swapExactTokensForETHSupportingFeeOnTransferTokens(  
    amountToSwap, 0, path, address(this), block.timestamp  
);
```

This swap increases the token supply in the pool and decreases the ETH supply, causing the KNX price to drop. After the swap, the reserved `amountToLiquify` tokens are now worth less ETH than before, while the received `amountETHLiquidity` reflects the pre-drop price. When `addLiquidityETH` is called, the router adjusts to the current reserves and typically consumes all tokens but only a portion of the ETH (or vice versa), returning the remainder to the contract.

A small residual amount of ETH or tokens remains unused after each liquidity addition cycle. These leftovers accumulate in the contract and are eventually consumed in future auto-swap cycles or recoverable via `recoverStuckETH`. The magnitude scales with swap size relative to pool reserves and is negligible for small swaps but may become noticeable for larger ones.

Assets:

- `contracts/KnoxNet.sol` [<https://github.com/Knoxnetofficial/knoxnet-contracts>]

Status: Accepted

Classification

Impact Rate: 1/5

Likelihood Rate: 1/5

Exploitability: Independent

Complexity: Simple

Severity: Info

Recommendations

Remediation: This is a known trade-off in auto-liquify token designs. Precise pre-calculation of the optimal split requires knowledge of the post-swap price, which creates a circular dependency. If higher precision is desired, the split ratio can be adjusted slightly below 50% to account for expected price impact, or the residual can be tracked and included in the next cycle explicitly.

Resolution: **Accepted:**

In **KnoxNet**, `_autoSwapBack` still reserves the liquidity-designated tokens using an exact 50/50 split, with half retained for liquidity addition and half swapped to ETH. No dynamic post-swap adjustment or alternative split formula has been introduced. The current implementation explicitly documents this behavior as intentional and notes that a more precise split would require knowledge of the post-swap price. The issue therefore remains unresolved and has been accepted as a design trade-off.

F-2026-16001 - Missing Cross-Parameter Validation in configureAutoSwap Allows Inefficient Auto-Swap Configurations - Info

Description:

In **KnoxNet**, the `configureAutoSwap` function validates `_denominator` in isolation but does not validate the relationship between `autoSwapTokenThreshold` and `autoSwapMinTokenBalance`. As a result, configurations may be accepted where the minimum balance required to trigger auto-swap is materially larger than the amount actually swapped per execution. This does not break functionality, but it allows inefficient or misleading auto-swap behavior.

In **KnoxNet**, `configureAutoSwap` derives the two key thresholds independently:

```
function configureAutoSwap(
    bool _enabled,
    uint256 _denominator,
    uint256 _minTokenBalance
) external onlyOwner {
    require(_denominator > 0 && _denominator <= 10000, "Denominator 1-10000");
    autoSwapEnabled = _enabled;
    autoSwapTokenThreshold = _totalSupply / _denominator;
    autoSwapMinTokenBalance = _minTokenBalance * (10 ** _decimals);
}
```

No consistency check is performed between these two resulting values. For example, `autoSwapMinTokenBalance` may be set to `1,000,000` tokens while `autoSwapTokenThreshold` is only `100,000` tokens. In that configuration, auto-swap triggers only after a large fee balance has accumulated, yet each execution swaps only a small fraction of that balance.

This behavior is valid under the current implementation, but it may be contrary to operator expectations if the two values are assumed to represent coordinated thresholds rather than unrelated independent bounds.

Auto-swap may trigger with a large contract token balance while processing only a comparatively small amount per execution. This can leave substantial fee balances idle for longer than expected and make configuration behavior harder to reason about. The issue affects operational efficiency and clarity rather than correctness or fund safety.

Assets:

- `contracts/KnoxNet.sol` [<https://github.com/Knoxnetofficial/knoxnet-contracts>]

Status:

Fixed

Classification

Impact Rate: 3/5

Likelihood Rate: 1/5

Exploitability: Dependent

Complexity: Simple

Severity: Info

Recommendations

Remediation:

- A relationship check should be added so that the minimum trigger balance and per-swap threshold remain internally coherent. At minimum, the contract should reject clearly inconsistent configurations or document that both values are intentionally independent.
- If independent configuration is desired, the expected interaction between the two values should be documented explicitly for operators.

Resolution:

Fixed in `c20e980` :

In **KnoxNet**, `configureAutoSwap` now validates the relationship between the derived trigger balance and the per-execution swap threshold. After computing both values, the function requires `autoSwapMinTokenBalance <= autoSwapTokenThreshold`, which prevents configurations where auto-swap would trigger only after a large balance has accumulated while swapping a materially smaller amount each time.

```
function configureAutoSwap(
    bool _enabled,
    uint256 _denominator,
    uint256 _minTokenBalance
) external onlyOwner {
    require(_denominator > 0 && _denominator <= 10000, "Denominator 1-10000");
    autoSwapEnabled = _enabled;
    autoSwapTokenThreshold = _totalSupply / _denominator;
    autoSwapMinTokenBalance = _minTokenBalance * (10 ** _decimals);
    require(autoSwapMinTokenBalance <= autoSwapTokenThreshold, "Min > threshold");
    emit AutoSwapConfigured(_enabled, autoSwapTokenThreshold, autoSwapMinToken
```

```
Balance);  
}
```

F-2026-16004 - Using block.timestamp as the Router Deadline

Removes Effective Expiry Protection - Info

Description:

In **KnoxNet**, router interactions use `block.timestamp` directly as the deadline parameter. Because the Uniswap V2 router only checks that `deadline >= block.timestamp` at execution time, a deadline equal to the current block timestamp is effectively always valid for the block in which the transaction is mined. This removes any meaningful expiry window and weakens the intended stale-transaction protection provided by the router interface.

In **KnoxNet**, router calls pass `block.timestamp` directly:

```
router.swapExactTokensForETHSupportingFeeOnTransferTokens(
    amountToSwap,
    0,
    path,
    address(this),
    block.timestamp
);
```

The same pattern is used in `addLiquidityETH` and `manualClogClearPercent`. In Uniswap V2, the deadline check is effectively:

```
require(deadline >= block.timestamp, "EXPIRED");
```

When `deadline` is set to `block.timestamp`, the check is satisfied whenever the transaction executes in the current block. This means the parameter does not provide a forward-looking expiration window and offers no protection against a transaction being mined later than intended at a materially changed market state.

The issue is most relevant for owner-triggered manual operations such as `manualClogClearPercent`, which may remain pending in the mempool before execution. For internal auto-swap actions that execute within an already-triggered user transaction, the practical risk is lower, but the deadline parameter still does not serve its intended purpose.

Assets:

- `contracts/KnoxNet.sol` [<https://github.com/Knoxnetofficial/knoxnet-contracts>]

Status:

Fixed

Classification

Impact Rate:	1/5
Likelihood Rate:	1/5
Exploitability:	Independent
Complexity:	Simple
Severity:	Info

Recommendations

Remediation: A forward-looking deadline should be used instead of `block.timestamp` directly. A short expiry window is generally sufficient.

Resolution: **Fixed in `c20e980` :**

In **KnoxNet**, router interactions no longer use `block.timestamp` directly as the deadline. A forward-looking expiry window is now applied through `ROUTER_DEADLINE_SECONDS`, and swap and liquidity operations pass `block.timestamp + ROUTER_DEADLINE_SECONDS` to the router. The stale-transaction protection described in the finding has therefore been restored.

```
uint256 constant ROUTER_DEADLINE_SECONDS = 300;
router.swapExactTokensForETHSupportingFeeOnTransferTokens(
    swapTokens,
    0,
    path,
    address(this),
    block.timestamp + ROUTER_DEADLINE_SECONDS
);
router.addLiquidityETH(value: ethAmount){
    address(this),
    tokenAmount,
    minToken,
    minETH,
    liquidityTaxReceiver,
    block.timestamp + ROUTER_DEADLINE_SECONDS
};
```

F-2026-16006 - Widespread Use of Magic Numbers Reduces Configuration Clarity and Auditability - Info

Description:

In **KnoxNet**, several important behavioral thresholds and fixed values are embedded directly as numeric literals rather than being expressed through named constants. This does not change runtime behavior, but it reduces readability and makes it harder to reason about the intent behind core limits and tax-related constraints. As a result, review, maintenance, and future parameter changes become more error-prone.

Multiple parts of **KnoxNet** rely on raw numeric literals with protocol significance, for example:

```
uint256 _maxBuyTxAmount = (_totalSupply * 1) / 100;
uint256 _maxSellTxAmount = (_totalSupply * 1) / 100;
uint256 _maxWalletSize = (_totalSupply * 1) / 100;
uint256 marketingBuyTaxBps = 500;
uint256 marketingSellTaxBps = 9000;
uint256 public autoSwapPerBlockLimit = 3;
uint256 public autoSwapSellCapPercent = 10;
```

The same pattern appears in validation logic:

```
require(
    _liquidityBuyTaxBps + _marketingBuyTaxBps <= 6500,
    "Buy tax cannot exceed 65%"
);
require(_denominator > 0 && _denominator <= 10000, "Denominator 1-10000");
```

Although these values may be legitimate design choices, the source code does not document them through descriptive constants such as `DEFAULT_MAX_TX_PERCENT`, `MAX_TAX_BPS`, or `DEFAULT_AUTO_SWAP_PER_BLOCK_LIMIT`. This makes protocol assumptions less explicit and increases the likelihood of inconsistent updates if values change in the future.

No incorrect on-chain behavior is introduced by this issue. However, code readability, auditability, and long-term maintainability are reduced because core economic and operational assumptions are encoded as unexplained literals. This increases the chance of operator or developer misunderstanding during future changes.

Assets:

- `contracts/KnoxNet.sol` [<https://github.com/Knoxnetofficial/knoxnet-contracts>]

Status: Mitigated

Classification

Impact Rate: 1/5

Likelihood Rate: 1/5

Exploitability: Independent

Complexity: Simple

Severity: Info

Recommendations

Remediation: Protocol-relevant numeric values should be extracted into named constants so that their purpose is explicit and reused consistently across the contract.

Resolution: **Mitigated in** `c20e980` :

In **KnoxNet**, several protocol-relevant numeric values have been extracted into named constants, including `BPS_DENOMINATOR`, `MIN_LIMIT_BPS`, `LIQUIDITY_SLIPPAGE_BPS`, and `ROUTER_DEADLINE_SECONDS`. This improves readability and makes part of the configuration logic more self-descriptive. However, multiple important thresholds and defaults are still embedded as raw literals, including the initial `1%` buy, sell, and wallet limits, the default tax values, the `6500` tax cap, the `3` swap-per-block limit, the `10` sell-cap percentage, and the `10000` bound used in input validation. The issue has therefore been reduced, but not fully resolved.

[F-2026-16007](#) - Multiple Percentage Conventions in the Same Contract Reduce Numeric Consistency and Increase Configuration Risk - Info

Description:

In **KnoxNet**, percentage-based parameters are expressed using multiple numeric conventions across the contract. Tax rates are represented in basis points using `taxDenominator`, transaction and wallet limits are configured through numerator/divisor fractions, and several operational caps use plain percentages out of `100`. This does not break arithmetic by itself, but it creates a fragmented configuration model that is harder to reason about and easier to misconfigure.

Different subsystems in **KnoxNet** use different percentage scales:

Tax rates use basis points relative to `taxDenominator`:

```
uint256 marketingBuyTaxBps = 500;
uint256 marketingSellTaxBps = 9000;
uint256 taxDenominator = 10000;
```

Transaction and wallet limits use a separate numerator/divisor fraction model:

```
_maxBuyTxAmount = (_totalSupply * buyNumerator) / divisor;
_maxSellTxAmount = (_totalSupply * sellNumerator) / divisor;
_maxWalletSize = (_totalSupply * numerator) / divisor;
```

Operational swap caps use plain percentages out of `100`:

```
uint256 public autoSwapSellCapPercent = 10;
uint256 maxSwapForThisSell = (amount * autoSwapSellCapPercent) / 100;
uint256 amountToSwap = (contractTokenBalance * percent) / 100;
```

These representations are all valid individually, but they are not unified. A caller configuring the contract must therefore remember which subsystem expects basis points, which expects a free-form divisor, and which expects percentages out of `100`. The issue is compounded by the presence of both fixed constants such as `6500` and dynamic denominators such as `taxDenominator`.

Administrative configuration becomes more error-prone, and protocol parameters become harder to compare consistently across subsystems. This increases operational friction and raises the likelihood of accidental misconfiguration.

Assets:

- contracts/KnoxNet.sol [<https://github.com/Knoxnetofficial/knoxnet-contracts>]

Status:

Accepted

Classification**Impact Rate:** 2/5**Likelihood Rate:** 1/5**Exploitability:** Dependent**Complexity:** Simple**Severity:** Info**Recommendations****Remediation:**

- Single percentage convention should be adopted wherever practical, preferably basis points with a fixed denominator, or the contract should clearly separate and document each unit system.
- Configuration functions should either normalize to a common convention or use strongly descriptive naming such as `sellCapPercent`, `buyTaxBps`, and `walletLimitBps` so that unit expectations are clear at the call site.

Resolution:**Accepted:**

In **KnoxNet**, externally accessible functions still use mixed amount-unit conventions. The `airdrop` function accepts whole-token values and scales them internally, `configureAutoSwap` mixes a ratio-based `_denominator` with a whole-token `_minTokenBalance` that is later scaled to base units, and the `maxBuyTxTokens`, `maxSellTxTokens`, and `maxWalletTokens` view functions return scaled-down whole-token values rather than raw base-unit amounts. No unified external amount convention has been introduced, and the inconsistency described in the finding remains present.

F-2026-16008 - Single-Step Ownership Transfer Increases the Risk of Irrecoverable Administrative Misassignment - Info

Description:

In **KnoxNet**, ownership transfer is implemented through the single-step **Ownable** pattern rather than a two-step acceptance flow. This means the current owner can immediately transfer control to any address, and the change becomes effective in the same transaction. As a result, an incorrect recipient address or an unprepared target account may irreversibly receive administrative control.

In **KnoxNet**, ownership transfer is inherited from the inlined **Ownable** implementation:

```
function transferOwnership(address newOwner) public virtual onlyOwner {
    require(newOwner != address(0), "Ownable: new owner is the zero address");
    _transferOwnership(newOwner);
}
```

No confirmation is required from the recipient. The new owner does not need to acknowledge the transfer, prove control over the address, or accept the role explicitly. If ownership is transferred to the wrong address, an incompatible contract, or an address that cannot be operated as intended, the administrative role is still reassigned immediately.

This is especially relevant in **KnoxNet** because the owner controls critical configuration such as tax parameters, limit settings, tax receivers, trusted operator assignments, and liquidity pool classification. While the pattern is not incorrect, it is less operationally safe than a two-step ownership handover.

Administrative control may be transferred irreversibly to an unintended or unusable address due to operator error. This can disrupt protocol administration and complicate recovery if the new owner is not able or willing to manage the contract correctly. The issue affects governance safety rather than core runtime correctness.

Assets:

- contracts/KnoxNet.sol [<https://github.com/Knoxnetofficial/knoxnet-contracts>]

Status:

Fixed

Classification

Impact Rate:

3/5

Likelihood Rate:	1/5
Exploitability:	Dependent
Complexity:	Simple
Severity:	Info

Recommendations

Remediation: A two-step ownership transfer pattern should be used so that the nominated new owner must explicitly accept the role. Using **Ownable2Step** or an equivalent pattern reduces the risk of accidental or irreversible ownership misassignment.

Resolution: **Fixed in** `c20e980` :

In **KnoxNet**, ownership transfer no longer follows a single-step handover. The `transferOwnership` function now sets a `_pendingOwner`, and control is reassigned only when the nominated address calls `acceptOwnership`. This introduces the required acceptance step and removes the risk of immediate ownership transfer to an unintended or unprepared address.

Disclaimers

Hacken Disclaimer

The smart contracts given for audit have been analyzed based on best industry practices at the time of the writing of this report, with cybersecurity vulnerabilities and issues in smart contract source code, the details of which are disclosed in this report (Source Code); the Source Code compilation, deployment, and functionality (performing the intended functions).

The report contains no statements or warranties on the identification of all vulnerabilities and security of the code. The report covers the code submitted and reviewed, so it may not be relevant after any modifications. Do not consider this report as a final and sufficient assessment regarding the utility and safety of the code, bug-free status, or any other contract statements.

While we have done our best in conducting the analysis and producing this report, it is important to note that you should not rely on this report only — we recommend proceeding with several independent audits and a public bug bounty program to ensure the security of smart contracts.

English is the original language of the report. The Consultant is not responsible for the correctness of the translated versions.

As part of Hacken's ongoing quality assurance process, we may conduct re-audits of select projects. These re-audits are performed independently from the original audit and are intended solely for internal quality control and improvement. Updated reports resulting from such re-audits will be shared privately with the respective clients and may be published on the Hacken website only with their explicit consent.

The sole authoritative source for finalized and most up-to-date versions of all reports remains the Audits section at <https://hacken.io/audits/>.

Technical Disclaimer

Smart contracts are deployed and executed on a blockchain platform. The platform, its programming language, and other software related to the smart contract can have vulnerabilities that can lead to hacks. Thus, the Consultant cannot guarantee the explicit security of the audited smart contracts.

Appendix 1. Definitions

Severities

When auditing smart contracts, Hacken is using a risk-based approach that considers **Likelihood**, **Impact**, **Exploitability** and **Complexity** metrics to evaluate findings and score severities.

Reference on how risk scoring is done is available through the repository in our Github organization:

[hknio/severity-formula](https://github.com/hacken/severity-formula)

Severity	Description
Critical	Critical vulnerabilities are usually straightforward to exploit and can lead to the loss of user funds or contract state manipulation.
High	High vulnerabilities are usually harder to exploit, requiring specific conditions, or have a more limited scope, but can still lead to the loss of user funds or contract state manipulation.
Medium	Medium vulnerabilities are usually limited to state manipulations and, in most cases, cannot lead to asset loss. Contradictions and requirements violations. Major deviations from best practices are also in this category.
Low	Major deviations from best practices or major Gas inefficiency. These issues will not have a significant impact on code execution.

Potential Risks

The "Potential Risks" section identifies issues that are not direct security vulnerabilities but could still affect the project's performance, reliability, or user trust. These risks arise from design choices, architectural decisions, or operational practices that, while not immediately exploitable, may lead to problems under certain conditions. Additionally, potential risks can impact the quality of the audit itself, as they may involve external factors or components beyond the scope of the audit, leading to incomplete assessments or oversight of key areas. This section aims to provide a broader perspective on factors that could affect the project's long-term security, functionality, and the comprehensiveness of the audit findings.

Appendix 2. Scope

The scope of the project includes the following smart contracts from the provided repository:

Scope Details	
Repository	https://github.com/Knoxnetofficial/knoxnet-contracts
Commit	748a36089e47f4b2d1bf0c6156663c7b1247be23
Remediation Commit	42b580b3d263748f4af5f3bcadd3e56d32554b1e
Whitepaper	-
Requirements	-
Technical Requirements	-

Asset	Type
contracts/KnoxNet.sol [https://github.com/Knoxnetofficial/knoxnet-contracts]	Smart Contract

Appendix 3. Additional Valuables

Additional Recommendations

The smart contracts in the scope of this audit could benefit from the introduction of automatic emergency actions for critical activities, such as unauthorized operations like ownership changes or proxy upgrades, as well as unexpected fund manipulations, including large withdrawals or minting events. Adding such mechanisms would enable the protocol to react automatically to unusual activity, ensuring that the contract remains secure and functions as intended.

To improve functionality, these emergency actions could be designed to trigger under specific conditions, such as:

- Detecting changes to ownership or critical permissions.
- Monitoring large or unexpected transactions and minting events.
- Pausing operations when irregularities are identified.

These enhancements would provide an added layer of security, making the contract more robust and better equipped to handle unexpected situations while maintaining smooth operations.

Frameworks and Methodologies

This security assessment was conducted in alignment with recognised penetration testing standards, methodologies and guidelines, including the [NIST SP 800-115 – Technical Guide to Information Security Testing and Assessment](#), and the [Penetration Testing Execution Standard \(PTES\)](#). These assets provide a structured foundation for planning, executing, and documenting technical evaluations such as vulnerability assessments, exploitation activities, and security code reviews. Hacken's internal penetration testing methodology extends these principles to Web2 and Web3 environments to ensure consistency, repeatability, and verifiable outcomes.